Master Thesis

Lightweight vs. Heavyweight CNN Architectures for Skin Lesion Classification

A Controlled Factorial Comparison of MobileNetV2, ResNet-50 and EfficientNet-B3 on HAM10000

Theerthankar Reddy Bandarla

SRH University of Applied Sciences

SRH University of Applied Sciences – Leipzig Campus

August 17, 2026

Master Thesis submitted in partial fulfilment of the requirements for the Degree of Master of Science in Computer Science with a focus on Big Data and Artificial Intelligence.

Matriculation Number: 100003667   Intake: Winter Semester 2024/25 (WiSe24/25)

Supervisor	Prof. Dr. Fakhteh Ghanbarnejad
Second Supervisor	Prof. Shamli Shamli

Abstract

Melanoma is survivable when found early and frequently fatal when it is not, so the practical value of automated skin lesion classification lies in triage rather than diagnosis. That places a constraint the accuracy literature rarely addresses: a triage tool has to run on the device in front of the patient. ResNet-50 serialises to 94.3 MB in the seven-class configuration used here; MobileNetV2 comes to 9.1 MB. The question is what that reduction costs.

The published record answers it poorly. Studies comparing architectures on HAM10000 vary the split protocol, augmentation, optimiser and schedule alongside the architecture, so a reported gap between two backbones conflates the backbone with the setup. Headline accuracy compounds the problem: HAM10000 is 66.9% benign nevi, so a model that answers nevus to every image scores 66.9% without recognising a single melanoma.

This thesis runs the comparison as a controlled experiment. A single pipeline holds the lesion-level split, augmentation, optimiser, batch size, schedule and hardware fixed, and varies only the architecture and the class-imbalance strategy. The result is a 2×3 factorial over MobileNetV2 and ResNet-50 crossed with augmentation-only, weighted cross-entropy and random oversampling, extended by a six-run EfficientNet-B3 ablation at two input resolutions and analysed in a separate statistical family. Performance is reported per class, with melanoma and basal cell carcinoma as the headline, and comparisons are tested with McNemar's paired test under Holm-Bonferroni correction.

Four findings emerge. MobileNetV2 achieved higher macro F1 than ResNet-50 under all three strategies, and under two of them the architectures were statistically indistinguishable on per-image correctness despite a ten-fold difference in parameter count. The best imbalance strategy depended on the architecture: weighted cross-entropy improved MobileNetV2's clinical recall while both remedies degraded ResNet-50's,

which Chapter 8 traces to optimisation stability rather than to class frequency. And selecting checkpoints on validation loss, the protocol this work inherited, cost up to 0.067 macro F1 and did so unevenly across architectures, which is a limitation of the design rather than a property of the models and is reported as such.

Keywords: skin lesion classification, HAM10000, class imbalance, MobileNetV2, ResNet-50, EfficientNet-B3, controlled factorial design, McNemar's test, deployable deep learning

Abbreviations

AKIEC	Actinic keratoses and intraepithelial carcinoma (Bowen's disease)
AUC	Area under the receiver operating characteristic curve
BCC	Basal cell carcinoma
BKL	Benign keratosis-like lesions
CE	Cross-entropy
CNN	Convolutional neural network
CPU	Central processing unit
DF	Dermatofibroma
F1	Harmonic mean of precision and recall
FN	False negative
FP	False positive
GPU	Graphics processing unit
HAM10000	Human Against Machine with 10000 training images
ISIC	International Skin Imaging Collaboration
MEL	Melanoma
NV	Melanocytic nevi
OvR	One-versus-rest
RQ	Research question
TP	True positive
VASC	Vascular lesions
VRAM	Video random-access memory

Chapter 1

Introduction and Motivation

1.1 Why Melanoma Detection Is a Timing Problem

Melanoma is survivable when it is caught early and frequently fatal when it is not. Five-year survival sits near 100% for lesions found while still localised and drops below 30% once the disease has metastasised [8]. Nothing about the tumour changes in that interval except how long it has been left alone. The clinical problem is therefore less about treatment than about how quickly a suspicious lesion reaches somebody qualified to look at it.

That is a resource problem. Dermatologists are unevenly distributed, waiting lists are long, and the people most at risk are often the least likely to present early. A triage tool that runs on a phone, in a general practice, or in a rural clinic without specialist cover does not need to replace a dermatologist. It needs to sort lesions well enough to move the worrying ones up the queue.

Deep learning has been able to do this in principle since Esteva et al. [8] trained an Inception-v3 network on roughly 129,000 clinical and dermoscopic images and matched board-certified dermatologists on biopsy-proven cases. The result has been reproduced in various forms often enough that the open question is no longer whether a convolutional network can classify skin lesions. It is which network you should actually deploy, and what that choice costs you.

1.2 The Gap Between Benchmark and Phone

Published accuracy is usually reported for architectures that were never meant to run on a handset. ResNet-50 [10] is the standard medical-imaging baseline and serialises to 94.3 MB in the seven-class configuration used in this thesis. MobileNetV2 [18], designed for exactly the on-device case, comes to 9.1 MB. Both figures are measured here rather than quoted, and the ratio is close to eleven to one.

An eleven-fold difference in size is not a rounding error. It decides whether a model ships inside an app or sits behind a network call, and a network call is precisely what an offline screening tool cannot depend on. So the practical question is narrow and answerable: what does the smaller network actually give up?

The published record answers this badly, for a reason worth stating plainly. Studies comparing architectures on HAM10000 tend to vary more than the architecture. Split protocol, augmentation, optimiser, batch size and epoch budget move together from paper to paper, so a reported gap between two backbones conflates the backbone with everything else that changed. Headline accuracy makes it worse. HAM10000 is 66.9% benign melanocytic nevi, so a model that answers *nevus* to every image scores 66.9% without ever recognising a melanoma. That number looks respectable next to much of the literature, and it is worthless.

1.3 What This Thesis Does

This thesis runs the comparison as a controlled experiment. One pipeline, one split, one optimiser, one schedule, and exactly two things allowed to vary: the architecture and the strategy used to handle class imbalance. That gives a 2×3 factorial over MobileNetV2 and ResNet-50 crossed with augmentation-only, weighted cross-entropy and random oversampling.

A third architecture, EfficientNet-B3 [14], was added after the original six runs were designed and executed. It is treated throughout as an ablation rather than as a third arm of the confirmatory design, and it is reported in its own statistical family. The distinction matters: the first six runs were specified before any results were seen, and folding a later addition into the same analysis would let a post-hoc decision change whether the pre-registered findings hold. EfficientNet-B3 is run twice, once at 224 pixels to match the other two backbones and once at its native 300, so that any difference can be attributed to the architecture rather than to the larger input it normally receives.

Performance is reported per class, with melanoma and basal cell carcinoma as the headline, because those are the classes a screening tool is answerable for. Overall accuracy appears in the results tables for completeness and is never used to decide anything.

1.4 Contributions

1. A controlled factorial on HAM10000 in which architecture and imbalance strategy are isolated from one another and from the rest of the pipeline, with the split hash recorded in every run so that any deviation is detectable rather than assumed absent.
2. Per-class precision, recall and F1 for all seven classes across every configuration, rather than the single accuracy figure that dominates the published record.
3. An EfficientNet-B3 ablation at two input resolutions, which separates the question of whether the architecture is better from the question of whether it was simply given more pixels.
4. Paired McNemar tests with Holm-Bonferroni correction, kept in two families so that pre-registered comparisons and later additions are not mixed.
5. Measured efficiency figures (parameter count, on-disk size, CPU latency) rather than the estimates usually quoted from the original architecture papers.
6. A reproducible implementation in which every reported number is generated from the stored run artefacts, so that the tables in this document cannot drift from the experiments that produced them.

1.5 Structure

Chapter 2 reviews the relevant literature and identifies what the existing record does and does not settle. Chapter 3 states the research gap and the three research questions, each with a pass condition fixed in advance. Chapter 4 describes HAM10000 and the lesion-level split protocol. Chapter 5 gives the methodology, including the code that was actually executed. Chapter 6 sets out the evaluation method and the statistical procedure. Chapter 7 reports results, Chapter 8 discusses them, and Chapter 9 concludes.

Chapter 2

Literature Review

2.1 Scope of the Review

This review covers four things: the benchmark this thesis uses, the architectures it compares, the class-imbalance remedies it crosses them with, and the statistical method it uses to decide whether two models actually differ. Work on segmentation, multi-modal fusion and explainability is noted where it bears on the comparison but is not the subject here.

2.2 HAM10000 as a Benchmark

HAM10000 was released by Tschandl et al. [22] as a deliberate answer to the shortage of public dermatoscopic data. It contains 10,015 images across seven diagnostic categories, collected over two decades from a Vienna clinic and an Australian practice, and it became the training set for the ISIC 2018 challenge [5]. Ground truth is more defensible than in most comparable sets: over half the cases are histopathologically confirmed rather than assigned by consensus.

Two properties matter for what follows. The first is that images are indexed by `lesion_id` as well as `image_id`, because the same lesion is often photographed more than once. Any split that ignores this puts near-duplicates on both sides of the train/test boundary. The second is the class distribution, which is severely long-tailed: melanocytic nevi account for roughly two thirds of the set while dermatofibroma accounts for about one percent. The ratio between the most and least frequent class is close to 58 to 1.

The literature is inconsistent about both. Reported HAM10000 accuracies range widely, and a meaningful part of that spread comes from protocol rather than method. Some studies split at image level, some at lesion level, and many do not say. Gao et al. [9] report 98.97% accuracy with a 284K-parameter network, which is difficult to reconcile with the difficulty other work reports on the same data unless the evaluation protocols differ substantially. This thesis takes the view that cross-paper accuracy comparison on HAM10000 is largely uninformative, and that the only comparisons worth trusting are ones run inside a single controlled pipeline.

2.3 Architectures

2.3.1 Residual networks as the default baseline

He et al. [10] introduced residual connections to make very deep networks trainable, and ResNet-50 has been the default medical-imaging backbone since. In dermatology specifically it appears constantly: Rezvantalab et al. [16] benchmark it among four architectures on dermoscopic data, and Shams et al. [20] use it as the reference point against lighter alternatives. Its ubiquity is the reason it serves as the heavyweight arm here. It is what a practitioner would reach for by default, so it is the right thing for a lightweight candidate to be measured against.

2.3.2 Efficiency-oriented designs

Howard et al. [11] introduced depthwise separable convolutions to cut the cost of a convolution by roughly an order of magnitude, and Sandler et al. [18] refined this into MobileNetV2 with inverted residuals and linear bottlenecks. The design target was explicitly on-device inference.

The dermatology literature has taken this up. Hussain et al. [1] propose S-MobileNet for seven-class HAM10000 classification, and Gao et al. [9] push further with a 284K-parameter module. Both report strong numbers. Neither reports a controlled comparison against a heavyweight baseline under identical conditions, which is the specific thing this thesis is built to provide.

2.3.3 EfficientNet and compound scaling

EfficientNet occupies a middle position: more capacity than MobileNetV2, far less than ResNet-50, with depth, width and input resolution scaled together rather than independently. Manole et al. [14] apply it to dermatological diagnosis and report 95.4% validation accuracy on a four-class problem. Shams et al. [20] compare ResNet-50, MobileNet and EfficientNet-B0 on DermNet and report 93%, 94% and 99% respectively, which would make EfficientNet the clear choice if the comparison were controlled.

It is not, and that is precisely the difficulty. EfficientNet variants are trained at their own native input resolutions, which increase with the variant index. B3 expects 300 pixels where MobileNetV2 and ResNet-50 expect 224. A study that gives EfficientNet its native resolution and the baselines theirs has varied two things at once, and cannot separate architectural merit from input size. This thesis addresses that directly by running B3 at both resolutions with everything else held fixed, which is what makes the pair E7/E10 informative rather than merely additional.

2.4 Class Imbalance

Buda et al. [3] provide the most systematic treatment, examining imbalance across MNIST, CIFAR-10 and ImageNet. Three findings carry over. Imbalance measurably degrades CNN performance; oversampling is generally the strongest of the standard remedies; and oversampling does not induce the overfitting in CNNs that it can in classical models. The last point is the one most often assumed rather than tested, and it is worth testing on a dataset as skewed as HAM10000.

Cui et al. [6] reweight by effective number of samples, generalising inverse-frequency weighting. That family of methods is what the weighted cross-entropy condition in this thesis implements, using $w_c = N/(K \cdot n_c)$. On the split used here that produces weights spanning 0.214 for nevi to 12.578 for dermatofibroma, a spread of roughly 59 to 1. Weights of that magnitude are worth treating as an intervention with side effects rather than a neutral correction.

Lin et al. [13] propose focal loss, down-weighting well-classified examples so training concentrates on hard ones. Le et al. [12] combine it with class weighting on HAM10000 and report 93% top-1 accuracy averaged across the seven classes. Focal loss is deliberately out of scope here, for reasons of factorial size rather than merit: adding it as a fourth strategy would take the design from twelve runs to sixteen. It is cited because a reviewer will ask, and because Le et al. is the closest published comparator to this work.

2.5 The Gap Le et al. Leaves Open

Le et al. [12] is the nearest neighbour to this thesis and therefore the most useful thing to be precise about. They vary the loss function on HAM10000 while holding the architecture fixed at a modified ResNet-50, and they report accuracy averaged over classes.

That leaves two things unanswered. The architecture never varies, so the study says nothing about what a lighter backbone would cost. And averaged accuracy hides the per-class picture, which is where the clinically relevant behaviour lives. A model can improve its average while getting worse at melanoma, and nothing in an averaged figure would reveal it.

The same pattern holds more broadly. Across the surveyed corpus, studies vary one factor at a time (architecture, or loss, or sampling) and report an aggregate metric. No published HAM10000 result crosses architecture with imbalance strategy in a single controlled design while reporting per-class results for the malignant categories.

2.6 Deciding Whether Two Models Differ

Comparing two classifiers on one test set is a statistical question, and the wrong test is common. Dietterich [7] evaluated five approximate tests for this purpose and found that when each model is trained once and evaluated on a single held-out set, McNemar's test is the only one with acceptable Type-I error. Tests assuming independent samples do not apply, because both models are evaluated on the same images.

That regime is exactly this thesis's. Each architecture-strategy combination is one training run producing one set of predictions. McNemar's test operates on the discordant pairs, the images one model classifies correctly and the other does not, and ignores the cases where both agree. Agreement carries no information about which model is better.

Running many such comparisons inflates the family-wise error rate, so correction is required. Holm-Bonferroni is used here as a uniformly more powerful alternative to plain Bonferroni that makes no additional assumptions about dependence between tests.

2.7 What the Review Establishes

Four things follow from the above and shape the rest of this thesis. Cross-paper accuracy comparison on HAM10000 is unreliable, so the comparison must happen inside one pipeline. Per-class reporting is necessary because aggregate metrics hide the clinically important behaviour. Input resolution is a confound whenever EfficientNet is compared against 224-pixel baselines, and must be controlled explicitly. And McNemar's test with correction across a declared family is the appropriate way to decide whether an observed difference is real.

Chapter 3

Research Gap, Questions and Objectives

3.1 Three Gaps

Chapter 2 identified three specific things the published record leaves open. They are stated here as gaps because each one is addressed by a design decision in Chapter 5.

Gap 1: no controlled architecture comparison. Studies comparing backbones on HAM10000 vary the split protocol, augmentation pipeline, optimiser, batch size and epoch budget alongside the architecture. A reported difference between two networks therefore confounds the network with the experimental setup. There is no published result that isolates the architecture.

Gap 2: no factorial crossing architecture with imbalance strategy. Imbalance remedies are studied one at a time, usually against a single fixed backbone. Whether the best remedy depends on which architecture it is applied to is an open question, and one that matters: if the answer is yes, then any recommendation of a remedy without reference to the backbone is incomplete.

Gap 3: aggregate metrics obscure the clinical picture. A model predicting nevus for every HAM10000 image achieves 66.9% accuracy while detecting no melanoma at all. Per-class F1 for the malignant categories is rarely the headline anywhere in the literature, and averaged accuracy can improve while melanoma detection degrades.

3.2 Research Questions

Each question below has a pass condition or selection rule fixed before any experiment was run. Fixing them in advance is what stops the analysis from drifting toward whatever the data happens to support.

RQ1: the deployability gap

Does the best MobileNetV2 configuration achieve a melanoma F1 within 0.05 of the best ResNet-50 configuration?

$$\left|\mathrm{F1}_{\mathrm{MEL}}(\text{best MobileNetV2}) - \mathrm{F1}_{\mathrm{MEL}}(\text{best ResNet-50})\right| \leq 0.05 \tag{3.1}$$

The threshold of 0.05 is a judgement, not a derivation, and is declared as such. It represents the largest difference in melanoma F1 that would still leave the lightweight model a defensible choice given roughly a tenfold reduction in size. A reviewer is entitled to disagree with the number; what matters is that it was set before the results existed rather than after.

"Best" is defined by the RQ2 criterion below, applied within each architecture. Section 7.4.1 in Chapter 7 also reports the result under the alternative reading in which "best" means highest macro F1, so that the conclusion can be checked against both definitions.

EfficientNet-B3 does not compete for RQ1. It was added after the confirmatory design was fixed, and allowing a later addition to displace one of the original arms would change what the pre-registered question was asking.

RQ2: which imbalance strategy, for which architecture

For each architecture, which of augmentation-only, weighted cross-entropy or random oversampling maximises mean recall on the two clinically significant classes?

$$\text{best}(a) = \arg\max_{s \in S} \frac{\mathrm{recall}_{\mathrm{MEL}}(a, s) + \mathrm{recall}_{\mathrm{BCC}}(a, s)}{2} \tag{3.2}$$

for architecture a over strategies S.

Recall rather than F1 or accuracy, because the two error types are not equally costly here. A melanoma classified as benign leaves untreated cancer. A benign lesion flagged as suspicious costs a dermatologist's time and a patient's anxiety. In a screening context that a specialist reviews downstream, the first error is much worse, and recall is the metric that penalises it.

This choice has a consequence that Chapter 7 reports rather than hides: the criterion can select a different configuration than macro F1 would. Both readings are given.

RQ3: what efficiency actually costs

How do the architectures compare on trainable parameter count, serialised model size, and CPU inference latency?

No pass threshold: the question is descriptive. The measurement conditions are fixed instead, and given in Chapter 6. Latency is measured on CPU because the deployment scenario under discussion is an offline device without a GPU, and a GPU figure would not speak to it.

3.3 The EfficientNet-B3 Ablation

EfficientNet-B3 was added at the request of the thesis supervisor after the original six runs were complete. It is handled as an ablation throughout, with two consequences.

First, it does not enter RQ1, as stated above. Second, its comparisons form a separate statistical family with its own correction, described in Section 6.6.2. Pooling twenty-one comparisons into one family would tighten the correction applied to the nine pre-registered ones, meaning a decision taken after seeing partial results could change whether the original findings reach significance. Keeping the families apart avoids that. It is a choice, and it is stated here rather than buried.

The ablation asks two things:

1. At matched input resolution, does EfficientNet-B3 outperform either incumbent? Runs E7 to E9 use 224 pixels, identical to the other six, so any difference is attributable to the architecture.

2. Does B3's native 300-pixel input account for its advantage, where one exists?

Runs E10 to E12 repeat E7 to E9 at 300 pixels with everything else unchanged, making each pair a controlled test of resolution alone.

The second question is the one the literature cannot currently answer, for the reason given in Section 2.3: published comparisons give each architecture its own native resolution and therefore vary two factors simultaneously.

3.4 Scope Exclusions

The following are excluded deliberately. Each is a real limitation and is revisited in Chapter 9.

Focal loss. Excluded to keep the factorial at twelve runs rather than sixteen. Cited in Chapter 2 as the method used by the closest published comparator [12].

Additional architectures. DenseNet, ConvNeXt and Vision Transformers appear in the surveyed corpus [23] and are not included. The question being answered is what a deployable network costs against a standard baseline, and each extra backbone dilutes that without sharpening it.

Ensembles and fusion. Several surveyed studies report gains from ensembling [4, 2]. An ensemble of n models occupies n times the space, which is the opposite of the deployment constraint motivating this work.

External validation. All runs use HAM10000 only. Cross-dataset generalisation to ISIC 2019 or 2020 would require a second data pipeline and would reintroduce exactly the protocol variation that Gap 1 is about.

Explainability. Grad-CAM [19] and related methods are not applied. This is a limitation rather than a considered exclusion: attention maps would help confirm the models attend to lesions rather than to acquisition artefacts.

Deployment itself. The thesis quantifies the trade-off. It does not build a mobile application, and reports no on-device measurements.

3.5 Objectives

1. Implement a single training pipeline in which architecture, imbalance strategy and input resolution are the only variables, with everything else held fixed and verifiable.
2. Execute the 2×3 confirmatory factorial and the six-run EfficientNet-B3 ablation on identical hardware from one shared data split.
3. Report per-class precision, recall and F1 for all seven classes in every configuration.
4. Answer RQ1 and RQ2 against the criteria fixed in Equations 3.1 and 3.2.
5. Measure RQ3 quantities directly rather than quoting them from the architecture papers.
6. Test every declared comparison with McNemar's test under Holm-Bonferroni correction, in the two families described above.
7. Persist per-run configuration, training history, test metrics and per-image predictions so that every number in this document is regenerable from stored artefacts.

Chapter 4

Data

4.1 HAM10000

All experiments use HAM10000 [22], obtained from Harvard Dataverse [21] rather than from a mirror, so the provenance is citable. The archive supplies 10,015 dermatoscopic images in two parts together with a metadata table.

Two practical details about that distribution are worth recording, because both will break a loader written against the more common Kaggle layout. The images arrive as two ZIP archives with the JPEGs at the root rather than inside named folders, and the metadata file is called `HAM10000_metadata` with no file extension despite containing ordinary comma-separated values. Both are handled explicitly in the data preparation notebook.

The metadata provides seven fields, of which three matter here: `lesion_id`, `image_id` and `dx`. The remaining fields (`dx_type`, `age`, `sex`, `localization`) are not used as model inputs. Every image is classified into one of seven diagnostic categories, listed in Table 4.1.

Table 4.1: The seven diagnostic categories in HAM10000, with observed counts.

Code	Diagnosis	Images	Share	Nature
nv	Melanocytic nevi	6,705	66.95%	Benign
mel	Melanoma	1,113	11.11%	**Malignant**
bkl	Benign keratosis-like lesions	1,099	10.97%	Benign
bcc	Basal cell carcinoma	514	5.13%	**Malignant**
akiec	Actinic keratoses / Bowen's	327	3.27%	Pre-malignant
vasc	Vascular lesions	142	1.42%	Benign
df	Dermatofibroma	115	1.15%	Benign
	Total	**10,015**	100%	

4.2 The Imbalance Problem, Quantified

The distribution in Table 4.1 is severely long-tailed. Melanocytic nevi outnumber dermatofibroma by 58.3 to 1. Figure 4.1 shows the shape.

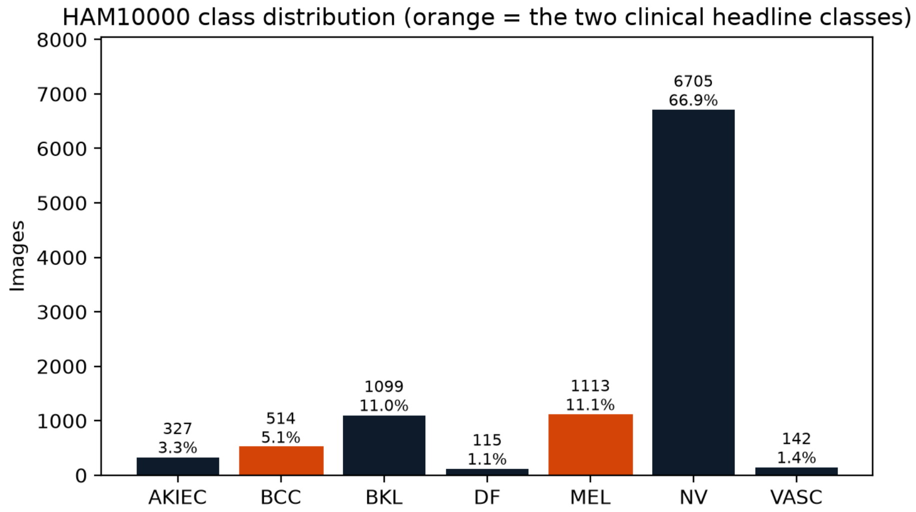

Figure 4.1: Class distribution across the full dataset. The two clinically significant classes, melanoma and basal cell carcinoma, are highlighted. Together they account for 16.2% of the data while carrying essentially all of the clinical risk.

The consequence is worth stating as a number rather than a warning. A classifier that ignores its input entirely and answers *nevus* every time achieves:

$$\frac{6{,}705}{10{,}015} = 66.95\% \text{ accuracy} \tag{4.1}$$

while detecting none of the 1,113 melanomas. That figure is not far below much of the accuracy reported in the HAM10000 literature. It is the reason this thesis treats overall accuracy as a diagnostic of dataset composition rather than of model quality, and reports per-class F1 as the primary metric instead.

4.3 Lesion-Level Splitting

HAM10000 photographs many lesions more than once. Of the 7,470 distinct lesions in the set, 1,956 contribute more than one image. Multiple photographs of the same lesion are near-duplicates: same patient, same site, same session, differing mainly in framing and illumination.

Splitting on `image_id` would therefore place near-identical images on both sides of the train/test boundary. The resulting test accuracy would partly measure memorisation of specific lesions rather than generalisation to new ones, and it would be inflated in a way that no amount of careful metric selection afterwards could detect.

The split is therefore performed on `lesion_id`, so every photograph of a given lesion lands in exactly one partition. It is stratified by diagnosis to preserve class proportions, uses an 80/10/10 ratio, and is seeded at 42.

Table 4.2: Partition sizes. The split is computed over lesions; image counts follow from which lesions land where, which is why the image ratios are not exactly 80/10/10.

Partition	**Lesions**	**Images**	**Share of images**
Train	5,976	8,012	80.0%
Validation	747	979	9.8%
Test	747	1,024	10.2%
Total	**7,470**	**10,015**	100%

The split is computed once, written to `runs/splits/seed42_lesion_stratified.json`, and hashed. Every subsequent run loads that file rather than recomputing, and records the hash in its own `config.json`. The hash for all runs reported in this thesis is `f35ac1de18678182`. A run that somehow saw different data would carry a different hash and be immediately identifiable, which converts an assumption into a check.

4.3.1 Leakage verification

Two assertions are executed in the data preparation notebook and must both hold before any training begins. No `lesion_id` may appear in more than one partition, and no `image_id` may appear in more than one partition. The second follows from the first but is checked independently, because it is the property that actually matters and a bug in the lesion-to-image mapping would violate it while leaving the first intact. Both pass with zero overlaps.

4.3.2 Stratification quality

Table 4.3 gives the resulting per-class distribution. Proportions are preserved to within roughly one percentage point in every class.

The test-set support figures deserve attention when reading later results. Melanoma

Table 4.3: Class composition of each partition. Test-set support is the figure that matters when reading per-class metrics in Chapter 7.

	Train		Validation		Test	
Class	n	%	n	%	n	%
nv	5,354	66.8	661	67.5	690	67.4
mel	892	11.1	106	10.8	115	11.2
bkl	896	11.2	98	10.0	105	10.3
bcc	403	5.0	58	5.9	53	5.2
akiec	261	3.3	33	3.4	33	3.2
vasc	115	1.4	12	1.2	15	1.5
df	91	1.1	11	1.1	13	1.3
Total	**8,012**		**979**		**1,024**	

has 115 test images and basal cell carcinoma has 53, so per-class metrics for those classes are computed over samples of that size. Dermatofibroma has 13. A single additional correct prediction moves dermatofibroma recall by roughly 7.7 percentage points, and per-class figures for the rarest classes should be read with that granularity in mind.

4.4 Class Weights

The weighted cross-entropy condition uses inverse-frequency weights computed on the training partition:

$$w_c = \frac{N}{K \cdot n_c} \tag{4.2}$$

where $N = 8{,}012$ is the training set size, $K = 7$ is the number of classes, and n_c is the count of class c in training. Table 4.4 gives the resulting values.

Table 4.4: Inverse-frequency class weights on the training partition.

Class	**Train** n	**Weight** w_c
nv	5,354	0.214
bkl	896	1.277
mel	892	1.283
bcc	403	2.840
akiec	261	4.385
vasc	115	9.953
df	91	12.578
Ratio, largest to smallest		**58.8**

A spread of 58.8 to 1 is large. A dermatofibroma training example contributes almost sixty times the gradient signal of a nevus example. This is the intended behaviour of inverse-frequency weighting, but it is an aggressive intervention rather than a neutral correction, and Chapter 8 returns to what it does to the runs that use it.

4.5 Preprocessing

Images are stored at 600×450 pixels. All are resized to the square input expected by the network, 224 pixels for the confirmatory runs and the matched-resolution ablation, 300 pixels for the native-resolution ablation, and normalised with ImageNet channel statistics [17] since all three backbones start from ImageNet-pretrained weights.

Training images additionally receive random horizontal flip, random vertical flip, rotation within $\pm 30°$, and colour jitter on brightness, contrast and saturation. Both flips are appropriate here because dermatoscopic images have no canonical orientation: a lesion photographed upside down is the same lesion. Colour jitter is bounded at 0.2 on each channel, deliberately modest, since colour carries genuine diagnostic information in dermatology and aggressive jitter would destroy signal rather than add invariance.

Validation and test images receive resize and normalisation only. No augmentation is applied at evaluation time, and no test-time augmentation is used anywhere in this thesis.

Chapter 5

Methodology

5.1 Design

The design is the simplest one that answers the research questions: a single training pipeline with exactly two pluggable variables, and one run per cell of the resulting grid. Table 5.1 gives the twelve runs.

Table 5.1: The twelve experiments. E1 to E6 are the confirmatory factorial, specified before any results were seen. E7 to E12 are the EfficientNet-B3 ablation, added afterwards and analysed separately.

Architecture	Input	Augmentation only	Weighted CE	Oversampling
Confirmatory factorial				
MobileNetV2	224	E1	E3	E5
ResNet-50	224	E2	E4	E6
Ablation				
EfficientNet-B3	224	E7	E8	E9
EfficientNet-B3	300	E10	E11	E12

The experiment registry is the single place where these definitions live. Nothing downstream hard-codes an architecture or a resolution; every script and notebook reads from this dictionary, which is what keeps the twelve runs from drifting apart.

```
EXPERIMENTS = {
    # --- confirmatory 2x3 factorial, 224px ---
    "E1":  {"architecture": "mobilenet_v2",    "strategy": "augmentation", "image_size": 224},
    "E2":  {"architecture": "resnet50",        "strategy": "augmentation", "image_size": 224},
    "E3":  {"architecture": "mobilenet_v2",    "strategy": "weighted_ce",  "image_size": 224},
```

```
    "E4":  {"architecture": "resnet50",        "strategy":
        "weighted_ce",  "image_size": 224},
    "E5":  {"architecture": "mobilenet_v2",    "strategy":
        "oversampling", "image_size": 224},
    "E6":  {"architecture": "resnet50",        "strategy":
        "oversampling", "image_size": 224},
    # --- EfficientNet-B3 ablation, matched 224px ---
    "E7":  {"architecture": "efficientnet_b3", "strategy":
        "augmentation", "image_size": 224},
    "E8":  {"architecture": "efficientnet_b3", "strategy":
        "weighted_ce",  "image_size": 224},
    "E9":  {"architecture": "efficientnet_b3", "strategy":
        "oversampling", "image_size": 224},
    # --- EfficientNet-B3 ablation, native 300px ---
    "E10": {"architecture": "efficientnet_b3", "strategy":
        "augmentation", "image_size": 300},
    "E11": {"architecture": "efficientnet_b3", "strategy":
        "weighted_ce",  "image_size": 300},
    "E12": {"architecture": "efficientnet_b3", "strategy":
        "oversampling", "image_size": 300},
}
```

Listing 5.1: The experiment registry, `src/ham10000/constants.py`.

5.2 What Varies and What Does Not

Table 5.2 is the experimental contract. Everything in the left column is identical across all twelve runs; only the right column changes.

Two entries in that table are less obvious than the rest and are worth justifying.

Dataloader workers belong in the contract. PyTorch seeds each dataloader worker from `base_seed + worker_id`, and the augmentation transforms draw from that per-worker generator. Changing the worker count therefore changes which random flip and rotation each image receives. It biases no condition, but it must be constant across runs for the comparison to remain controlled, which is why it is listed here rather than treated as an implementation detail.

Table 5.2: Experimental contract.

Held identical across all twelve	Varied
Split: lesion-level, stratified 80/10/10, seed 42, hash `f35ac1de18678182`	**Architecture**: MobileNetV2, ResNet-50, EfficientNet-B3
Augmentation: HFlip, VFlip, rotation $\pm 30^\circ$, colour jitter 0.2	**Imbalance strategy**: augmentation-only, weighted CE, oversampling
Normalisation: ImageNet channel statistics	**Input resolution**: 224 or 300 (EfficientNet-B3 only)
Initialisation: ImageNet-pretrained, classifier head reset to 7 outputs	
Optimiser: Adam, lr $= 10^{-3}$, $\beta = (0.9, 0.999)$	
Batch size: 32	
Schedule: max 50 epochs, early stopping on validation loss, patience 10	
Dataloader workers: 16	
Seed: 42	
Hardware: one NVIDIA RTX A4500 for every run	

Batch size is fixed even where memory would allow more. The 300-pixel runs have headroom on the GPU used. Increasing their batch size would speed them up and would also make them incomparable with the other nine, so it was not done.

5.3 Architectures

All three backbones are loaded from `torchvision` with ImageNet-pretrained weights. Only the final linear layer is replaced, mapping to seven outputs. No layers are frozen; every run fine-tunes end to end.

```python
def build_model(architecture: str, num_classes: int = 7) -> nn.Module:
    arch = architecture.lower()

    if arch == "mobilenet_v2":
        model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.IMAGENE
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, num_classes)
        return model

    if arch in ("resnet50", "resnet_50"):
```

```
        model =
            models.resnet50(weights=models.ResNet50_Weights.IMAGENET1K_V1)
        model.fc = nn.Linear(model.fc.in_features, num_classes)
        return model

    if arch in ("efficientnet_b3", "efficientnet-b3"):
        model =
            models.efficientnet_b3(weights=models.EfficientNet_B3_Weights.I
        model.classifier[1] =
            nn.Linear(model.classifier[1].in_features,
            num_classes)
        return model

    raise ValueError(f"Unknown architecture: {architecture}.
        Expected one of {SUPPORTED}.")
```

Listing 5.2: Backbone construction, `src/ham10000/models.py`. For MobileNetV2 and EfficientNet-B3 the classifier is a `Sequential(Dropout, Linear)`, so replacing index 1 leaves the dropout at index 0 in place.

Table 5.3 reports the structural properties of the three networks as instantiated, counted from the constructed modules rather than quoted from the original papers.

Table 5.3: Structural properties, counted from the built models with a seven-class head.

Architecture	**Trainable params**	**Weight layers**	**Blocks**	**Dropout**
MobileNetV2	2,232,839	53	17 inverted residual	$p = 0.2$
ResNet-50	23,522,375	54	16 bottleneck	none
EfficientNet-B3	10,706,991	131	26 MBConv	$p = 0.3$

Two observations from that table shape later interpretation.

First, ResNet-50's 54 weight layers exceed the 50 in its name. The `torchvision` implementation contains 53 convolutional modules because four are 1×1 projection convolutions on the shortcut path at stage transitions, which the canonical count omits. The figure is reported as counted so that it can be reconciled with the code.

Second, EfficientNet-B3 is by far the deepest at 131 weight layers while holding less than half of ResNet-50's parameters. It is a deep, narrow network where ResNet-50 is shallower and wider. Chapter 7 shows this has consequences for inference latency that parameter count alone does not predict.

5.3.1 An asymmetry in dropout

The three backbones do not carry the same regularisation. MobileNetV2 ships with a dropout layer at $p = 0.2$ before its classifier, EfficientNet-B3 with $p = 0.3$, and ResNet-50 with none at all. Because only the final `Linear` is replaced, each network retains whatever its reference implementation provides.

This is left as-is deliberately, and it is a defensible choice rather than an oversight, but it is also a genuine asymmetry between the arms and is declared as such. The comparison is intended to be between these networks as they are published and as anyone would actually deploy them. Matching dropout across all three would compare three variants that exist nowhere else. The cost is that a small part of any advantage held by the two networks with dropout could be attributable to the regularisation rather than to the architecture. Chapter 9 records the matched-dropout ablation as future work; it is a one-line change.

5.4 Imbalance Strategies

```
def make_train_loader(dataset, train_labels, strategy, batch_size=32,
                      num_workers=2, pin_memory=True):
    strategy = strategy.lower()
    if strategy == "augmentation":
        loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
                            num_workers=num_workers, pin_memory=pin_memory)
        return loader, torch.nn.CrossEntropyLoss()

    if strategy == "weighted_ce":
        class_weights = inverse_frequency_weights(train_labels)
        loader = DataLoader(dataset, batch_size=batch_size, shuffle=True,
                            num_workers=num_workers, pin_memory=pin_memory)
        return loader, torch.nn.CrossEntropyLoss(weight=class_weights)

```

```
    if strategy == "oversampling":
        weights = sample_weights_for_oversampling(train_labels)
        sampler = WeightedRandomSampler(weights,
           num_samples=len(weights), replacement=True)
        loader = DataLoader(dataset, batch_size=batch_size,
           sampler=sampler,
                            num_workers=num_workers,
                               pin_memory=pin_memory)
        return loader, torch.nn.CrossEntropyLoss()

    raise ValueError(f"Unknown imbalance strategy: {strategy}")
```

Listing 5.3: The three strategies, `src/ham10000/imbalance.py`. Each returns a dataloader and the loss function that goes with it.

Augmentation-only (E1, E2, E7, E10). Standard cross-entropy with shuffled sampling. The augmentation pipeline of Section 4.5 still applies, as it does everywhere, but nothing is done about imbalance. This is the no-remediation control against which the other two are read.

Weighted cross-entropy (E3, E4, E8, E11). Per-class weights from Equation 4.2, passed to the loss. Sampling is unchanged. The model sees the same images in the same proportions; the gradient contribution of each is rescaled.

Random oversampling (E5, E6, E9, E12). A `WeightedRandomSampler` draws with replacement at probability proportional to $1/n_c$, so an epoch contains roughly balanced class frequencies while the loss stays unweighted. Rare-class images recur many times per epoch; common-class images are frequently skipped. This is the condition that tests Buda et al.'s [3] finding that oversampling does not induce overfitting in CNNs.

The distinction between the last two is worth keeping clear, because they are often treated as interchangeable. Weighted cross-entropy changes how much each example counts. Oversampling changes how often each example is seen. They arrive at a similar objective by different routes and, as Chapter 7 shows, do not produce the same result.

5.5 Training

```
# No weight decay, no LR schedule, no gradient clipping. Held identical
# across all twelve runs, and disclosed in the methodology chapter.
optimizer = torch.optim.Adam(model.parameters(), lr=config.lr, betas=(0.9, 0.999))
```

Listing 5.4: Optimiser construction, `src/ham10000/train.py`.

Adam at 10^{-3} with default β values, and nothing else. That is a deliberate omission and needs stating explicitly, because the absence of these is as much a design decision as their presence would be.

5.5.1 Regularisation applied, and not applied

Four mechanisms restrain overfitting: the augmentation pipeline, early stopping on validation loss with patience 10, ImageNet pre-training as a strong prior on the weights, and the architecture-specific dropout of Section 5.3.1.

What is not applied: no weight decay, since `weight_decay` retains its default of zero and there is no L_2 penalty; no learning-rate schedule; no gradient clipping; no label smoothing. These omissions are uniform, so they confound no comparison between conditions. They remain a real limitation, and Chapter 8 argues they are the most plausible explanation for the optimisation behaviour observed in the ResNet-50 runs.

5.5.2 Early stopping and the epoch budget

Training runs to a maximum of 50 epochs with early stopping on validation loss at patience 10. The best checkpoint by validation loss is the one evaluated on test.

The patience value was revised upward from an initial 5 during piloting, after inspecting the validation curves rather than any test-set number. ResNet-50 was still finding new best losses at the point where patience 5 terminated it, meaning the original budget was cutting off a run that had not converged. The revision was applied uniformly to all twelve conditions, so the comparison remains controlled. It is nonetheless a deviation from the originally registered protocol and is disclosed here rather than absorbed silently.

5.5.3 Resumability and configuration drift

Every epoch writes a checkpoint. A run interrupted by a dropped connection resumes from the last completed epoch rather than restarting, which matters when twelve runs share one rented GPU session.

Resumption introduces a hazard that is easy to miss: a checkpoint written under one configuration can be resumed under another, splicing two protocols into a single history file with nothing in the output to indicate it happened. The training loop therefore compares the stored configuration against the current one and refuses to continue on any mismatch.

```python
_MUST_MATCH = ("architecture", "strategy", "image_size", "batch_size",
               "lr", "num_workers", "seed", "split_hash")
old = ckpt.get("config", {})
drift = {
    k: (old.get(k), run_config.get(k))
    for k in _MUST_MATCH
    if k in old and old.get(k) != run_config.get(k)
}
if drift:
    raise RuntimeError(
        f"{config.experiment_id} has a checkpoint from a different configuration:\n"
        f"Resuming would mix both protocols in one run."
    )
```

Listing 5.5: Configuration drift guard, `src/ham10000/train.py`.

5.6 Compute Environment

All twelve runs execute on one rented NVIDIA RTX A4500 (20,470 MiB) under PyTorch 2.8.0 with CUDA 12.8 and torchvision 0.23.0 [15]. Using a single GPU for every run removes hardware as a source of variation: cuDNN kernel selection is not identical across GPU architectures even with deterministic mode enabled, so runs split across different cards would not be strictly comparable.

Peak GPU memory was measured for a real training step in each configuration before the runs began, to confirm the card was adequate rather than assume it.

Table 5.4: Peak GPU memory for a training step at batch size 32, measured on the RTX A4500.

Configuration	Peak	Experiments
MobileNetV2 @224	2.6 GB	E1, E3, E5
ResNet-50 @224	3.1 GB	E2, E4, E6
EfficientNet-B3 @224	5.8 GB	E7, E8, E9
EfficientNet-B3 @300	10.2 GB	E10, E11, E12

The 300-pixel runs set the ceiling at 10.2 GB, comfortably inside the card. The dataset was staged into a RAM-backed filesystem for training after profiling showed that reading individual JPEGs from the network-attached volume, not GPU computation, was the limiting factor: 45 images per second from the network mount against 241 from RAM. This affects throughput only. The bytes, their order and the seeds are unchanged, so it has no bearing on any reported result.

5.7 Reproducibility

Seeds are fixed at 42 for Python, NumPy and PyTorch, and cuDNN runs in deterministic mode with benchmarking disabled.

```python
def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

Listing 5.6: Seeding, `src/ham10000/train.py`.

Each run persists five artefacts: `config.json` with the fully resolved configuration including the split hash, GPU name and library versions; `history.csv` with per-epoch training and validation curves; `test_metrics.json` with the full metric set and confusion matrix; `test_predictions.csv` with a per-image prediction, required for the paired statistical tests; and the best checkpoint by validation loss.

Every table and figure in Chapter 7 is generated from those artefacts by script rather than transcribed. A number in this document that disagreed with the stored results would require the generating code to be wrong, not merely a typo, which is a stronger guarantee than proofreading provides.

Chapter 6

Evaluation Method

6.1 Why Not Accuracy

Equation 4.1 established that a constant classifier scores 66.95% on this dataset. Any metric that a useless model can score well on is not a metric that distinguishes useful models from each other. Accuracy is reported in the results tables because omitting it would look evasive, and it is used to decide nothing.

The metrics that do the work are per-class, and two classes carry the weight. Melanoma is the class whose misdiagnosis is lethal. Basal cell carcinoma is the most common malignancy in the set. Together they are 16.2% of the data and essentially all of the clinical risk.

6.2 Metrics

For each class c, with TP_c, FP_c and FN_c the true positives, false positives and false negatives:

$$\text{precision}_c = \frac{TP_c}{TP_c + FP_c} \tag{6.1}$$

$$\text{recall}_c = \frac{TP_c}{TP_c + FN_c} \tag{6.2}$$

$$\text{F1}_c = 2 \cdot \frac{\text{precision}_c \cdot \text{recall}_c}{\text{precision}_c + \text{recall}_c} \tag{6.3}$$

The asymmetry between the two error types is the whole reason precision and recall are reported separately rather than only as their harmonic mean. For melanoma, FN

is a cancer sent home as benign. FP is a healthy patient referred for a second look. In a screening context where a dermatologist reviews the flagged cases, these are not comparable costs, and a single F1 figure conceals which way a given configuration is erring.

Three aggregate metrics are also reported. Macro F1 is the unweighted mean of per-class F1, giving each class equal weight regardless of frequency, which is what makes it informative here where accuracy is not. Balanced accuracy is the mean of per-class recall. Macro AUC-OvR is the one-versus-rest area under the ROC curve averaged over classes, computed from predicted probabilities rather than hard labels, and is therefore insensitive to the decision threshold.

A 7×7 confusion matrix is reported per run. Aggregates say how much a model got wrong; the confusion matrix says what it confused with what, which is the part a clinician would ask about.

6.3 Selection Criteria

The criteria fixed in Chapter 3 are restated here in the form the analysis code implements.

For RQ2, the score for each run is the unweighted mean of recall on the two clinical classes, and the best configuration per architecture is the arg-max:

```
def rq2_score(exp: str) -> float:
    return (report(exp, "mel", "recall") + report(exp, "bcc",
        "recall")) / 2
```

Listing 6.1: The RQ2 criterion, `scripts/build_tables.py`.

For RQ1, the winners under that criterion are compared on melanoma F1 against the 0.05 threshold. Because "best" is a definitional choice, Chapter 7 also reports the comparison under the alternative reading where best means highest macro F1. If the two readings disagree about whether RQ1 passes, that is a finding about the fragility of the criterion and is reported as one.

RQ1 is computed over the confirmatory experiments only, which the code enforces rather than leaves to discipline:

```python
# RQ1 is defined over the confirmatory pair only. B3 is an ablation and does
# not replace either incumbent in the pre-registered question.
conf = [e for e in exps if e in CONFIRMATORY_EXPERIMENTS]
mob = [e for e in conf if EXPERIMENTS[e]["architecture"] == "mobilenet_v2"]
res = [e for e in conf if EXPERIMENTS[e]["architecture"] == "resnet50"]
```

Listing 6.2: RQ1 restricted to the confirmatory factorial, `scripts/build_tables.py`.

6.4 Frequency Tiers

Per-class metrics across seven classes are detailed but hard to read as a whole. As a secondary view, classes are grouped into a majority tier (nv, bkl, mel) and a minority tier (akiec, vasc, df) by training frequency, and mean F1 is reported for each.

This is a compression of information already present in the per-class table, included because it makes one specific question legible: whether a configuration is buying minority-class performance at the expense of the majority. A run whose minority-tier mean exceeds its majority-tier mean has inverted the usual ordering, which is a strong signal about what the imbalance remedy did.

6.5 Efficiency Measurement

RQ3 quantities are measured, not quoted. Parameter count is the sum over parameters with `requires_grad`. Model size is the serialised `state_dict` in megabytes. Latency is wall-clock time for a single forward pass at batch size 1, measured on CPU after ten warm-up iterations, reported as mean and standard deviation over 100 timed runs.

CPU rather than GPU because the deployment scenario motivating this thesis is an offline device with no accelerator. A GPU latency figure would be easy to obtain and would not speak to the question.

EfficientNet-B3 is measured at both 224 and 300 pixels, since latency scales with input area and a single figure would misrepresent it.

```python
dummy = torch.randn(1, 3, image_size, image_size,
    device=device)
times_ms = []
with torch.no_grad():
    for _ in range(WARMUP_RUNS):
        model(dummy)
    for _ in range(TIMED_RUNS):
        start = time.perf_counter()
        model(dummy)
        times_ms.append((time.perf_counter() - start) * 1000)
```

Listing 6.3: Latency measurement, `scripts/benchmark_efficiency.py`.

Timing on shared or virtualised hardware is noisy, and the measurement host is recorded alongside the numbers so the figures can be read with that in mind. Where the standard deviation is large relative to the mean, the comparison between architectures remains informative because all are measured under identical conditions, but the absolute values should not be treated as a specification.

6.6 Statistical Testing

6.6.1 McNemar's test

Following Dietterich [7], comparisons use McNemar's paired test. For two models evaluated on the same test set, the 2×2 contingency table counts images by whether each model classified them correctly:

Table 6.1: McNemar contingency table. Only b and c enter the test.

	B correct	**B wrong**
A correct	a	b
A wrong	c	d

The null hypothesis is $b = c$: the two models disagree equally often in both directions. Cells a and d are discarded, which is the essential property. Images both models handle identically carry no information about which is better, and on a dataset where two thirds of images belong to an easy majority class, a test that counted them would be dominated by agreement.

The exact binomial form is used when $b + c < 25$, where the chi-squared approximation is unreliable, and the continuity-corrected chi-squared form otherwise. The selection is automatic rather than chosen per comparison:

```
discordant = a_only + b_only
# Exact binomial when the discordant count is small; the
    chi-squared
# approximation is unreliable there.
use_exact = discordant < 25
result = mcnemar([[both_right, a_only], [b_only, both_wrong]],
                 exact=use_exact, correction=not use_exact)
```

Listing 6.4: Test variant selection, `scripts/statistical_tests.py`.

Predictions are joined on `image_id` rather than by row position, and the test asserts that both runs agree on ground truth before proceeding. A mismatch there would indicate the two runs saw different splits, which would invalidate the pairing.

6.6.2 Two families

Twenty-one comparisons are declared, in two families corrected independently.

The **confirmatory family** contains nine comparisons over E1 to E6: three cross-architecture within strategy, and six cross-strategy within architecture. These were specified before any results existed.

The **exploratory family** contains twelve comparisons involving EfficientNet-B3: six against the incumbents at matched resolution, three across strategies within B3 at 224, and three resolution pairs.

```
CONFIRMATORY = [
    # architecture held constant within strategy
    ("E1", "E2"), ("E3", "E4"), ("E5", "E6"),
    # strategy varied within architecture
    ("E1", "E3"), ("E1", "E5"), ("E3", "E5"),
    ("E2", "E4"), ("E2", "E6"), ("E4", "E6"),
]

EXPLORATORY = [
    # B3 at 224 against each incumbent, strategy held constant
    ("E7", "E1"), ("E7", "E2"),
    ("E8", "E3"), ("E8", "E4"),
```

```
    ("E9", "E5"), ("E9", "E6"),
    # strategy varied within B3 at 224
    ("E7", "E8"), ("E7", "E9"), ("E8", "E9"),
    # resolution: same architecture, same strategy, 224
        against 300
    ("E7", "E10"), ("E8", "E11"), ("E9", "E12"),
]
```

Listing 6.5: The two declared families, `scripts/statistical_tests.py`.

Holm-Bonferroni correction is applied within each family at $\alpha = 0.05$. Holm is used rather than plain Bonferroni because it is uniformly more powerful while making no additional assumptions.

Correcting separately is a decision with a consequence, and it should be visible rather than buried. Pooling all twenty-one into one family would apply a harsher correction to the nine pre-registered comparisons, so a decision taken after the confirmatory runs had completed could determine whether those earlier findings reach significance. Separate families prevent that. The cost is that the exploratory family's own error rate is controlled only within itself, which is the appropriate treatment for comparisons that were not planned in advance.

The implementation refuses to correct over a partial family:

```
    """Holm-correct within one family.

    Skips the family entirely if any member is missing rather
        than silently
    correcting over a subset: dropping comparisons changes the
        correction and
    would quietly make the surviving ones look more
        significant than they are.
    """
```

Listing 6.6: Refusing partial correction, `scripts/statistical_tests.py`.

6.6.3 What a non-significant result means here

A comparison that fails to reach significance is not a null result to be passed over. It states that the two configurations disagree about equally often in both directions.

When one of them is roughly a tenth the size of the other, that is a substantive finding about deployability rather than an absence of one, and Chapter 7 reports non-significant comparisons on the same footing as significant ones.

Chapter 7

Results

7.1 Status of the Experimental Programme

All twelve runs are complete: the confirmatory factorial (E1 to E6) in Sections 7.2 to 7.7, and the EfficientNet-B3 ablation (E7 to E12) in Section 7.10.

Every number in this chapter is generated from the stored run artefacts by `scripts/build_thesis_t` and `scripts/make_figures.py` rather than transcribed, so the tables cannot drift from the experiments that produced them.

All twelve runs carry split hash `f35ac1de18678182`, confirming they trained and were evaluated on identical partitions. Tables covering all twelve include the ablation rows; the confirmatory analysis in Sections 7.4 and 7.5 is computed over E1 to E6 only, for the reason given in Section 3.3.

7.2 Headline Metrics

One pattern dominates Table 7.1: **MobileNetV2 achieves higher macro F1 than ResNet-50 under every one of the three imbalance strategies**. The margins are 0.037 under augmentation, 0.138 under weighted cross-entropy and 0.154 under oversampling. The lightweight network is never the weaker of the pair on this metric, which is the opposite of what the ordering of model sizes would predict.

Accuracy tells a different and less useful story. E1 and E2 sit within 0.004 of each other at roughly 0.80, close to the 0.669 a constant classifier achieves and close to one another despite a 0.037 gap in macro F1. This is the behaviour Section 6.1 anticipated: accuracy on this dataset is largely a report on how well a model handles nevi.

Table 7.1: Test-set metrics. Epochs is the number completed before early stopping. Best value in each column in bold.

Exp	Configuration	Epochs	Accuracy	Bal. acc.	Macro F1	Macro AUC
E1	MobileNetV2 + aug	38	0.801	0.671	0.664	0.958
E2	ResNet-50 + aug	46	0.797	0.592	0.628	0.946
E3	MobileNetV2 + weighted CE	34	0.711	0.744	0.622	0.945
E4	ResNet-50 + weighted CE	50	0.699	0.557	0.483	0.929
E5	MobileNetV2 + oversampling	30	0.795	0.712	0.681	0.954
E6	ResNet-50 + oversampling	16	0.725	0.656	0.527	0.930
E7	EfficientNet-B3 @224 + aug	15	0.805	0.637	0.674	0.955
E8	EfficientNet-B3 @224 + weighted CE	21	0.740	0.687	0.614	0.943
E9	EfficientNet-B3 @224 + oversampling	20	0.805	0.761	0.704	0.958
E10	EfficientNet-B3 @300 + aug	21	**0.835**	0.672	0.700	0.962
E11	EfficientNet-B3 @300 + weighted CE	26	0.805	**0.770**	**0.738**	**0.964**
E12	EfficientNet-B3 @300 + oversampling	18	0.832	0.695	0.714	0.954

E4 is the weakest run in the set on every aggregate metric, and it is also the only run to exhaust the epoch budget. It ran the full 50 epochs without early stopping triggering, meaning validation loss was still improving, or at least not failing to improve for ten consecutive epochs, when the ceiling was reached. Its results should be read as those of a run that had not converged. Chapter 8 takes this up.

At the other extreme, E6 stopped at 16 epochs, less than a third of E4's budget, and still outperformed it on macro F1 by 0.044.

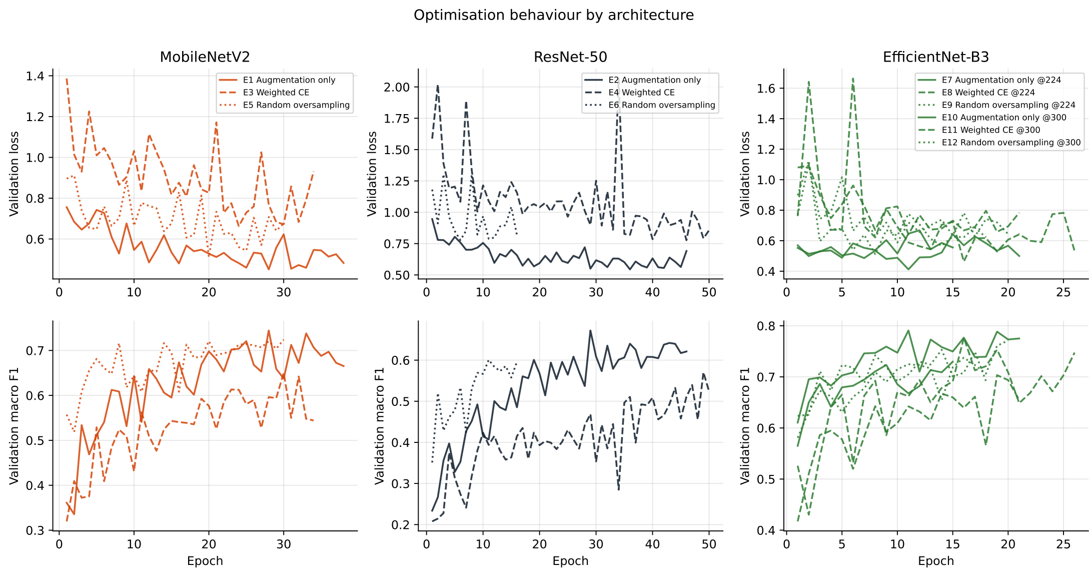

Figure 7.1: Validation loss (top) and validation macro F1 (bottom) by epoch, grouped by architecture. MobileNetV2 descends smoothly; ResNet-50 is visibly noisier under the shared learning rate.

7.3 Per-Class Results

Table 7.2: Per-class F1 for all seven classes. The two clinical classes are in bold.

Exp	AKIEC	BCC	BKL	DF	MEL	NV	VASC
E1	0.605	**0.697**	0.631	0.696	**0.473**	0.905	0.643
E2	0.500	**0.589**	0.600	0.632	**0.480**	0.903	0.690
E3	0.520	**0.631**	0.552	0.489	**0.442**	0.836	0.882
E4	0.353	**0.275**	0.508	0.349	**0.450**	0.856	0.593
E5	0.605	**0.619**	0.634	0.538	**0.506**	0.900	0.968
E6	0.433	**0.542**	0.484	0.436	**0.254**	0.877	0.667
E7	0.667	**0.713**	0.570	0.632	**0.537**	0.906	0.696
E8	0.505	**0.634**	0.583	0.625	**0.376**	0.869	0.710
E9	0.605	**0.667**	0.645	0.750	**0.575**	0.901	0.788
E10	0.632	**0.667**	0.670	0.783	**0.590**	0.926	0.636
E11	0.703	**0.772**	0.647	0.727	**0.564**	0.893	0.857
E12	0.712	**0.674**	0.664	0.667	**0.561**	0.921	0.800

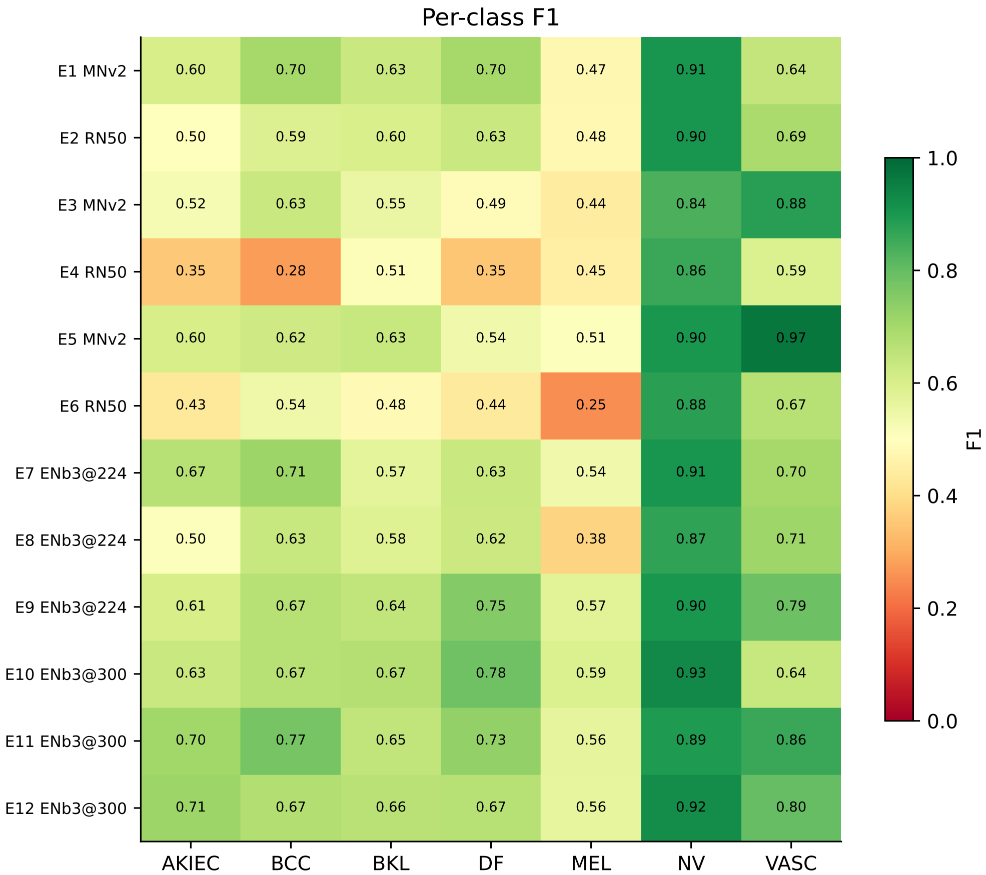

Figure 7.2: Per-class F1 across the confirmatory factorial. NV is handled well by every configuration; the variation that distinguishes them lives in the rarer classes.

Nevi are classified consistently well everywhere, between 0.836 and 0.905, which is what a class with 690 test images and two thirds of the training data should look like. The disagreement between configurations lives almost entirely in the other six classes.

Melanoma F1 spans 0.254 (E6) to 0.506 (E5), a factor of two across configurations of the same two architectures on the same data. That spread is the practical case for reporting per-class metrics: nothing in Table 7.1 distinguishes E6's melanoma performance from E2's as sharply as this does.

7.3.1 Precision and recall on the clinical classes

Table 7.3: Precision, recall and F1 on the two clinical classes. The final column is the RQ2 selection criterion, the unweighted mean of the two recall values.

	Melanoma			BCC			
Exp	**P**	**R**	**F1**	**P**	**R**	**F1**	**RQ2 score**
E1	0.486	0.461	0.473	0.679	0.717	0.697	0.589
E2	0.580	0.409	0.480	0.559	0.623	0.589	0.516
E3	0.340	0.635	0.442	0.603	0.660	0.631	0.648
E4	0.382	0.548	0.450	0.407	0.208	0.275	0.378
E5	0.484	0.530	0.506	0.682	0.566	0.619	0.548
E6	0.667	0.157	0.254	0.537	0.547	0.542	0.352
E7	0.486	0.600	0.537	0.750	0.679	0.713	0.640
E8	0.354	0.400	0.376	0.667	0.604	0.634	0.502
E9	0.503	0.670	0.575	0.673	0.660	0.667	0.665
E10	0.627	0.557	0.590	0.694	0.642	0.667	0.599
E11	0.466	0.713	0.564	0.721	0.830	0.772	**0.772**
E12	0.585	0.539	0.561	0.795	0.585	0.674	0.562

Table 7.3 is where the trade-off becomes explicit, and it justifies reporting precision and recall separately rather than only F1.

E3 attains the highest melanoma recall of any run at 0.635 and the lowest melanoma precision at 0.340. It finds nearly two thirds of melanomas, and roughly one in three of the lesions it calls melanoma actually is one. E6 is the mirror image: melanoma precision of 0.667, the best in the set, on recall of 0.157. When E6 says melanoma it is usually right, and it says melanoma for only 18 of the 115 melanomas present.

For a screening tool those two failure modes are not equivalent. E6 misses 97 melanomas. E3 misses 42 and refers a larger number of benign lesions for review.

7.4 RQ1: The Deployability Gap

The RQ2 criterion selects E3 as the best MobileNetV2 configuration and E2 as the best ResNet-50 configuration. Applying Equation 3.1:

$$\left|\mathrm{F1}_{\mathrm{MEL}}(E3) - \mathrm{F1}_{\mathrm{MEL}}(E2)\right| = \left|0.442 - 0.480\right| = 0.037 \leq 0.05$$

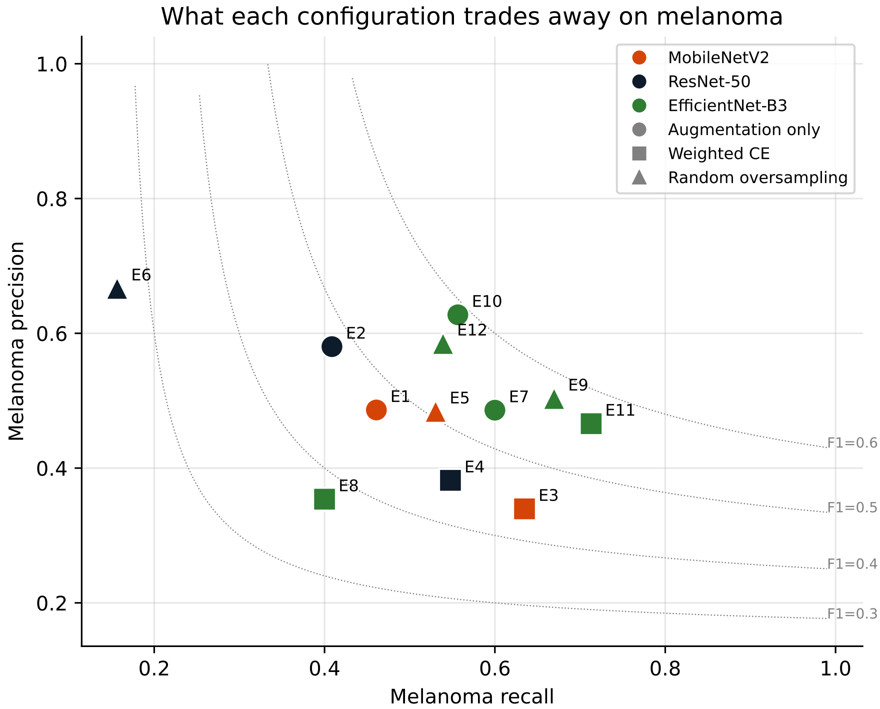

Figure 7.3: Melanoma precision against recall, with iso-F1 contours. Configurations sit on a trade-off curve rather than being uniformly better or worse.

RQ1 passes. The gap of 0.037 falls inside the threshold fixed in advance.

7.4.1 Robustness to the definition of "best"

Because "best" is a definitional choice, the comparison was repeated under the alternative reading in which best means highest macro F1. That selects E5 for MobileNetV2 and E2 for ResNet-50, giving a gap of 0.027, which also passes and does so by a wider margin.

The conclusion therefore does not depend on which definition is used. Under the recall-based criterion the lightweight model is 0.037 behind on melanoma F1; under the macro-F1 criterion it is 0.027 ahead. Both are inside the threshold.

7.5 RQ2: Best Strategy per Architecture

Weighted cross-entropy is the best strategy for MobileNetV2. Augmentation-only is the best strategy for ResNet-50. The answer is architecture-dependent, which is what Gap 2 of Section 3.1 asked about, and it means a recommendation of an imbalance remedy without reference to the backbone would be incomplete.

Table 7.4: RQ2 criterion by architecture and strategy. Best per architecture in bold.

	Augmentation only	Weighted CE	Oversampling
MobileNetV2	0.589 (E1)	**0.648 (E3)**	0.548 (E5)
ResNet-50	**0.516 (E2)**	0.378 (E4)	0.352 (E6)
EfficientNet-B3	0.640 (E7)	0.502 (E8)	0.665 (E9)

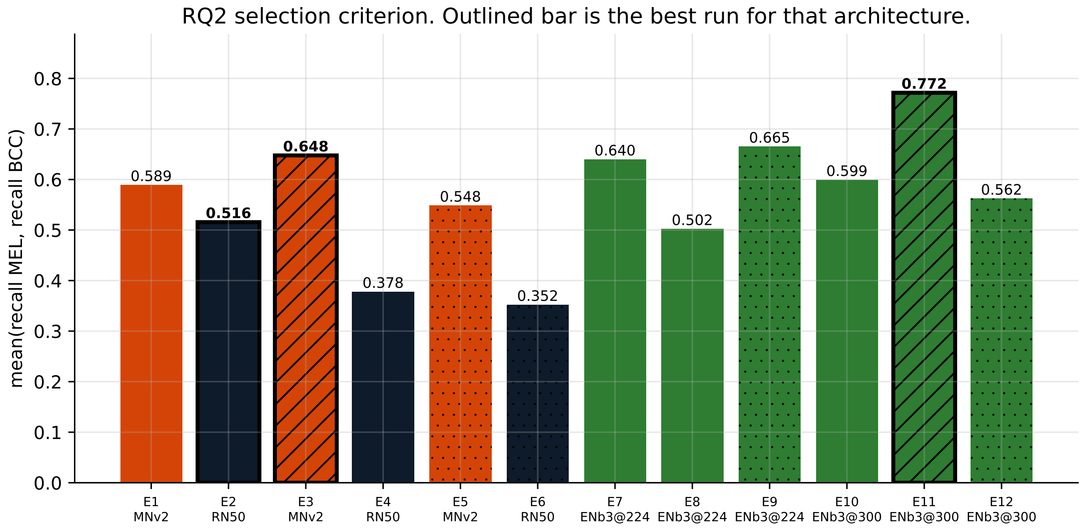

Figure 7.4: RQ2 criterion by run. Outlined bars are the best configuration for that architecture.

The result is more striking than the table alone conveys. Both remediation strategies *reduce* the criterion for ResNet-50: 0.516 with no remediation falls to 0.378 with weighted cross-entropy and 0.352 with oversampling. For MobileNetV2, weighted cross-entropy raises it from 0.589 to 0.648 while oversampling lowers it to 0.548.

Interventions designed to help rare classes made ResNet-50 worse at exactly the classes they target. Chapter 8 argues this is a consequence of optimisation instability rather than of the remedies themselves.

7.5.1 The metric-dependence of "best"

Ranking by macro F1 instead gives a different answer. For MobileNetV2 the ordering becomes E5 (0.681), E1 (0.664), E3 (0.622), so oversampling wins where weighted cross-entropy won on recall. For ResNet-50 the ordering is E2 (0.628), E6 (0.527), E4 (0.483), so augmentation-only wins under both criteria.

The MobileNetV2 disagreement is a genuine precision/recall trade-off rather than noise. E3 buys melanoma recall of 0.635 at precision 0.340; E5 balances at 0.530 and 0.484.

F1 prefers the balance, the clinical criterion prefers the recall, and both are defensible depending on whether a false negative or a false positive is more costly. This thesis commits to the recall reading for the reason given in Section 3.2, and reports both.

7.6 Frequency Tiers

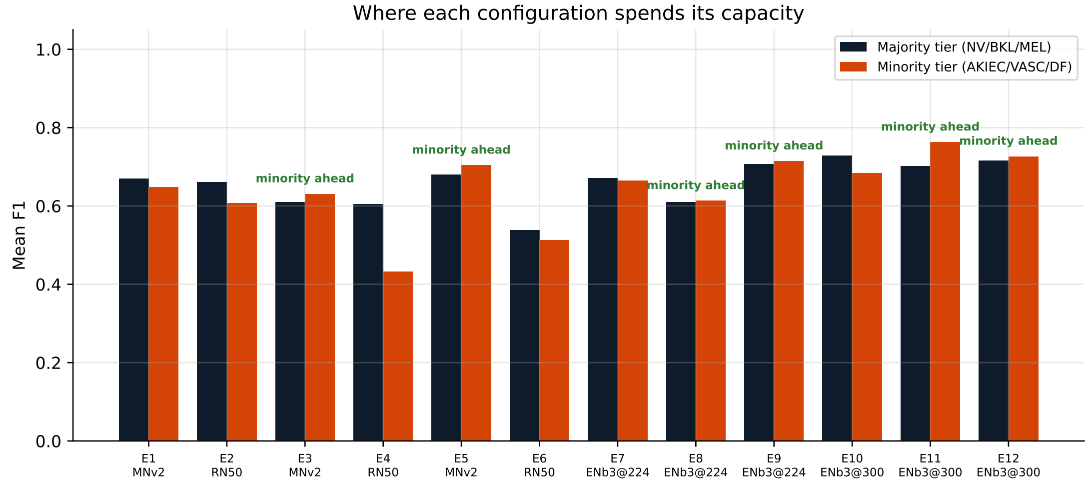

Figure 7.5: Mean F1 over the majority tier (NV, BKL, MEL) against the minority tier (AKIEC, VASC, DF).

The tier view separates configurations that improved rare-class performance from those that merely traded one class against another. It corroborates the per-class table without adding independent evidence, since both derive from the same predictions.

7.7 Statistical Testing

Table 7.5: McNemar's test, confirmatory family, Holm-Bonferroni corrected at $\alpha = 0.05$. 5 of 9 comparisons reach significance.

Comparison	p (Holm)	Significant	Favours
E1 vs E2	1	No	E1
E3 vs E4	1	No	E3
E5 vs E6	6.07e−6	Yes	E5
E1 vs E3	1.62e−8	Yes	E1
E1 vs E5	1	No	E1
E3 vs E5	6.84e−9	Yes	E5
E2 vs E4	2.30e-11	Yes	E2
E2 vs E6	8.26e−7	Yes	E2
E4 vs E6	0.368	No	E6

Three results in Table 7.5 matter.

Table 7.6: McNemar's test, exploratory family, Holm-Bonferroni corrected at $\alpha = 0.05$. 4 of 12 comparisons reach significance.

Comparison	p **(Holm)**	**Significant**	**Favours**
E7 vs E1	1	No	E7
E7 vs E2	1	No	E7
E8 vs E3	0.222	No	E8
E8 vs E4	0.0651	No	E8
E9 vs E5	1	No	E9
E9 vs E6	2.00e−7	Yes	E9
E7 vs E8	1.88e−5	Yes	E7
E7 vs E9	1	No	E9
E8 vs E9	9.16e−6	Yes	E9
E7 vs E10	0.0848	No	E10
E8 vs E11	1.92e−5	Yes	E11
E9 vs E12	0.19	No	E12

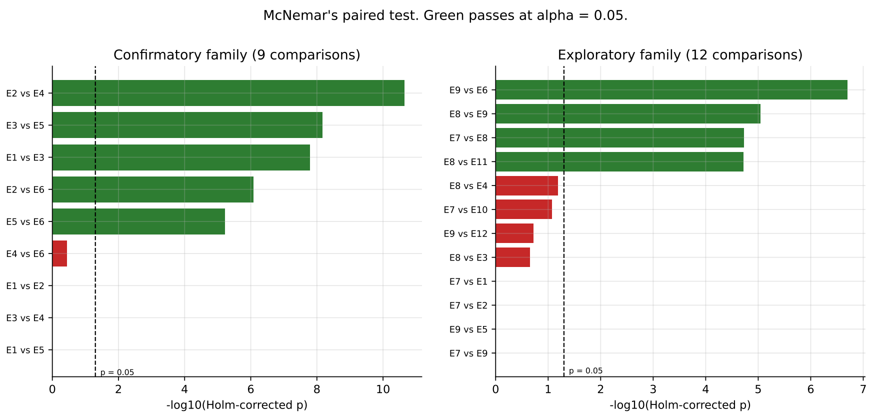

Figure 7.6: Holm-corrected p-values, confirmatory family. Green passes at $\alpha = 0.05$.

The architectures are statistically indistinguishable under two of three strategies. E1 vs E2 and E3 vs E4 both fail to reach significance. Under augmentation and under weighted cross-entropy, MobileNetV2 and ResNet-50 disagree about equally often in both directions. Given that MobileNetV2 is roughly a tenth of ResNet-50's size, an inability to distinguish them is a substantive finding rather than an absence of one.

Where they do differ, the lightweight model wins. E5 vs E6 is significant at $p = 6.07 \times 10^{-6}$ in favour of MobileNetV2. In no comparison does ResNet-50 significantly outperform MobileNetV2.

Strategy matters more than architecture for ResNet-50. Both E2 vs E4 and E2 vs E6 are significant and both favour augmentation-only, with p values several orders of magnitude below threshold. The imbalance remedies did measurable harm to ResNet-50, and the effect is larger than any architecture effect observed.

7.8 Training Cost

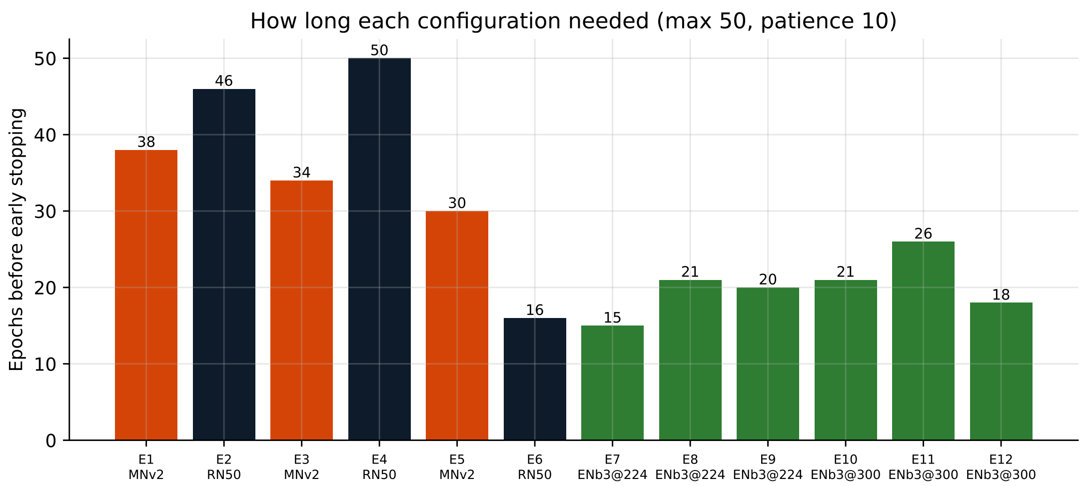

Figure 7.7: Epochs completed before early stopping. E4 alone exhausted the 50-epoch budget.

Epoch counts vary by more than a factor of three, from 16 (E6) to 50 (E4). MobileNetV2 runs averaged 34 epochs at roughly 21.5 seconds each; ResNet-50 runs averaged 37 epochs at roughly 30.8 seconds each. The lightweight network was cheaper per epoch and required no more epochs, so it was cheaper to train in total under every strategy.

E4 reaching the ceiling is a limitation of that run rather than a property of the configuration, and its numbers carry an asterisk throughout: it is the one result in this chapter produced by a model that had not finished converging.

7.9 Confusion Structure

The dominant off-diagonal mass in every run lies between nevi, melanoma and benign keratoses. These three are the classes a dermatologist also finds hardest to separate on dermoscopy alone, so the models are failing along clinically sensible lines rather than at random.

The melanoma-to-nevus cell is the one with clinical consequence, since it is a malignancy classified as benign. It is largest in E6, consistent with that run's melanoma recall of 0.157.

7.10 EfficientNet-B3 Ablation

The ablation asks two questions: whether EfficientNet-B3 outperforms either incumbent at matched input resolution, and whether its native 300-pixel input accounts for any advantage it shows. Both are answered against the twelve exploratory comparisons of Listing 6.5, corrected within their own family.

7.10.1 At matched resolution

Table 7.7: Macro F1 at 224 pixels, all three architectures, by strategy. Every model in this table receives an identically sized input.

Strategy	MobileNetV2	ResNet-50	EfficientNet-B3
Augmentation only	0.664 (E1)	0.628 (E2)	**0.674 (E7)**
Weighted CE	**0.622 (E3)**	0.483 (E4)	0.614 (E8)
Random oversampling	0.681 (E5)	0.527 (E6)	**0.704 (E9)**

EfficientNet-B3 takes the highest macro F1 under two of the three strategies and loses narrowly to MobileNetV2 under the third. E9 at 0.704 is the strongest 224-pixel run in the study.

The statistical picture is more restrained. Of the six comparisons pitting B3 against an incumbent at matched resolution, only one reaches significance after Holm correction: E9 against E6 ($p = 2.00 \times 10^{-7}$), where the ResNet-50 run it beats is the weakest of

Confusion matrices (row-normalised, counts annotated)

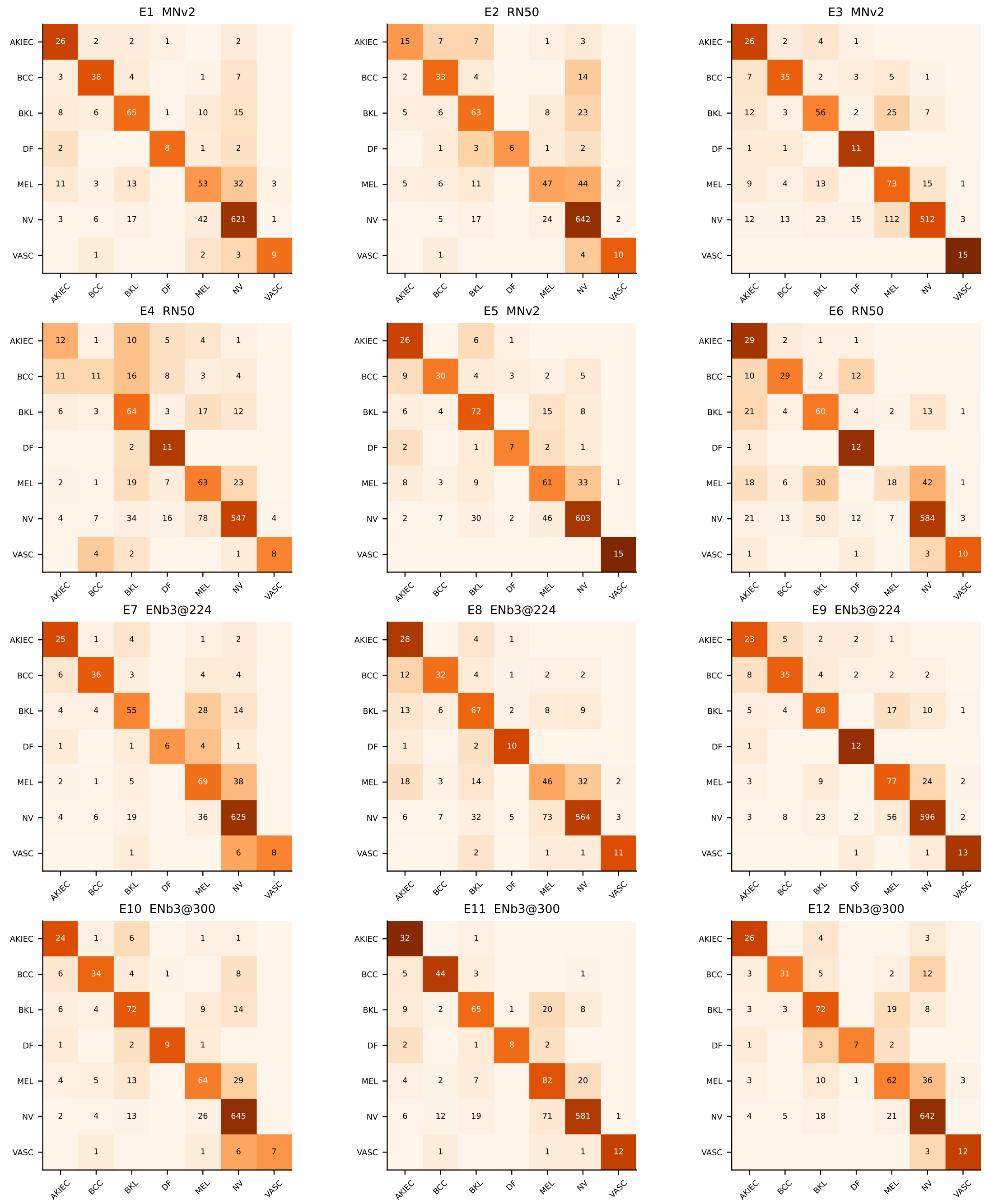

Figure 7.8: Confusion matrices, row-normalised with counts annotated.

that architecture's three. Against MobileNetV2 specifically, all three comparisons are non-significant (E7 vs E1 and E9 vs E5 both at $p = 1.000$, E8 vs E3 at $p = 0.222$).

So at matched input size EfficientNet-B3 and MobileNetV2 cannot be distinguished on per-image correctness, while B3 holds 4.80 times the parameters and 4.76 times the disk footprint. That is the same shape of result as RQ1, one step further along: the capacity that separates these architectures is not what limits performance on this dataset.

7.10.2 Does the native resolution explain it?

Table 7.8: The three resolution pairs. Each row holds architecture, strategy, split, seed and schedule fixed and varies only the input size, so the difference is attributable to resolution alone.

Strategy	224 (macro F1)	300 (macro F1)	Δ	p (Holm)	Significant
Augmentation only	0.674 (E7)	0.700 (E10)	$+0.026$	0.085	No
Weighted CE	0.614 (E8)	0.738 (E11)	$+0.123$	1.92e−05	**Yes**
Random oversampling	0.704 (E9)	0.714 (E12)	$+0.010$	0.190	No

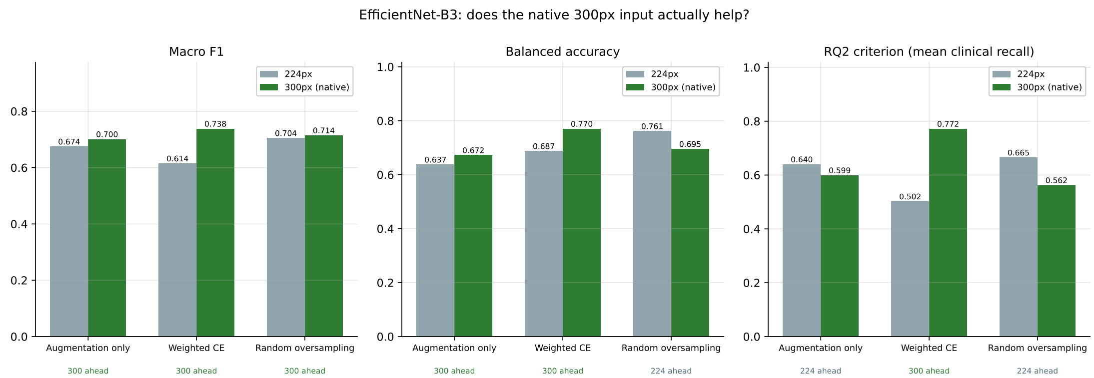

Figure 7.9: EfficientNet-B3 at 224 against 300 pixels, by strategy. Everything other than the input size is identical within each pair.

The native resolution helps in all three pairs, but by amounts that differ by an order of magnitude, and only one difference survives correction.

Under augmentation-only and oversampling the gain is 0.026 and 0.010 respectively, and neither reaches significance. Under weighted cross-entropy the gain is 0.123, and it does ($p = 1.92 \times 10^{-5}$). **Input resolution matters for EfficientNet-B3 only in combination with weighted cross-entropy.**

The pattern is consistent with the interpretation offered for ResNet-50 in Section 7.5. Weighted cross-entropy is the highest-variance condition in the study: it multiplies the loss on a dermatofibroma example by 12.578 against 0.214 for a nevus. E8, the 224-pixel weighted-CE run, is the weakest B3 configuration at 0.614. Raising the input resolution appears to give the network enough additional signal per image to cope with that reweighting, and the effect is largest precisely where the run was struggling. Where B3 was already doing well, more pixels added little.

This is the question the published record cannot answer. Comparisons that give EfficientNet its native resolution and the baselines theirs vary architecture and input size together (Section 2.3), so a reported EfficientNet advantage cannot be attributed to either. Here the two are separated, and the answer is that resolution contributes materially in one of three conditions and negligibly in the other two.

7.10.3 Why this does not change the deployment recommendation

E11 attains the highest macro F1 (0.738), the highest balanced accuracy (0.770) and the highest RQ2 clinical-recall score (0.772) of any run in this thesis. Taken on accuracy alone it is the best configuration produced here.

It is also the least deployable. At 300 pixels on the ARM64 host, EfficientNet-B3 runs at 126.5 ms per image against MobileNetV2's 14.2 ms, a factor of 8.9, and occupies 43.18 MB against 9.07 MB (Table 7.9). The thesis's question is not which architecture reaches the highest number; it is what a deployable architecture costs. On that question B3 answers in the wrong direction on both axes that matter for an offline mobile tool.

The ablation therefore leaves the conclusion of Section 7.4 intact and adds a qualification to it. MobileNetV2 remains the recommended deployment choice, and the reason is now better supported: it is statistically indistinguishable from both a network ten times its size and a network five times its size, while being the fastest and smallest of the three. EfficientNet-B3 would be the right choice for a server-side deployment where latency and footprint are not binding, and Chapter 9 records that as the boundary of what this result supports.

Table 7.9: RQ3 efficiency. Latency is milliseconds per image at batch size 1 on one CPU thread, mean of 100 timed runs after 10 warm-up iterations, with the ratio to MobileNetV2 in brackets. Measured on two CPU architectures because the ordering of ResNet-50 and EfficientNet-B3 is not the same on both.

Architecture	Params	Size (MB)	ARM64	x86-64
MobileNetV2	2,232,839	9.07	14.2 (1.0x)	19.5 (1.0x)
ResNet-50	23,522,375	94.30	31.1 (2.2x)	133.2 (6.8x)
EfficientNet-B3 @224	10,706,991	43.18	89.5 (6.3x)	66.9 (3.4x)
EfficientNet-B3 @300	10,706,991	43.18	126.5 (8.9x)	116.4 (6.0x)

7.11 RQ3: Efficiency

Measurements were taken on two machines, each pinned to a single CPU thread and each otherwise idle. Standard deviations are small relative to the means (0.2–5.2 ms on means of 14–133 ms), so the comparisons are not dominated by scheduling noise.

MobileNetV2 holds 10.53 times fewer parameters than ResNet-50 and occupies 10.40 times less disk, at 9.07 MB against 94.30 MB. That is the deployability case in one line: a 9 MB model ships inside a mobile application, and a 94 MB one does not comfortably. Parameter count and disk size are properties of the architecture and do not vary by host; latency, as the next section shows, is a different matter.

7.11.1 Parameter count does not predict latency

The latency columns do not follow the parameter column, and they do not agree with each other either. Both facts matter.

On both hosts MobileNetV2 is the fastest of the three, and on both it is faster by a margin far smaller than its parameter advantage would suggest. It holds 10.53 times fewer parameters than ResNet-50 but is only 2.2 times faster on ARM64. Parameter count is a poor predictor of inference time.

The stronger result is that **the ordering of ResNet-50 and EfficientNet-B3 reverses between the two CPU architectures**. On ARM64, ResNet-50 runs at 31.1 ms and EfficientNet-B3 at 89.5 ms, so B3 is 2.9 times slower despite holding 2.2 times fewer parameters. On x86-64 the same two models run at 133.2 ms and 66.9 ms, so B3 is 2.0 times *faster*. The same weights, the same input, opposite conclusions.

The mechanism is arithmetic intensity. ResNet-50's dense convolutions perform many operations per byte of memory traffic; the depthwise separable convolutions in MobileNetV2 and EfficientNet-B3 perform few. Whether that trade is favourable depends on where the host binds. The ARM machine used here reaches dense convolutions through heavily optimised vendor kernels and handles ResNet-50 well, while the single-threaded x86-64 host has no comparable advantage and pays ResNet-50's full operation count.

Two consequences follow for this thesis. First, MobileNetV2's advantage is the only part of the efficiency picture that is robust across hosts, which strengthens rather than weakens the deployability argument. Second, any efficiency claim about a specific pair of heavier architectures is a claim about a specific CPU, and reporting it without naming the host is not meaningful. Since the deployment target motivating this work is a mobile device, and mobile devices are ARM, the ARM64 column is the relevant one for the thesis's own argument. The x86-64 column is reported because omitting it would have left a conclusion stated more broadly than the evidence supports.

EfficientNet-B3's 131 weight layers against ResNet-50's 54 (Table 5.3) contribute on both hosts: depth is sequential, and a deep narrow network offers less work per layer to parallelise.

7.11.2 Resolution scales latency sub-linearly

EfficientNet-B3 at 300 pixels processes 1.79 times the pixels of the same network at 224 and takes 1.41 times as long on ARM64 and 1.74 times on x86-64. The gap between those two figures is fixed overhead that does not scale with input area: layer launches, weight loading and memory allocation.

The practical reading is that B3's native resolution costs roughly 40 to 75% more inference time than running it at 224, depending on host. Whether that is worth paying depends on whether the larger input buys accuracy, which is what runs E10 to E12 are designed to establish and which Section 7.10 will report.

7.11.3 RQ3 answered

MobileNetV2 is the most deployable of the three by every measure taken: smallest at 9.07 MB, fewest parameters at 2.23 million, and fastest on both CPU architectures tested. It is the only one of the three whose advantage is robust to the choice of host.

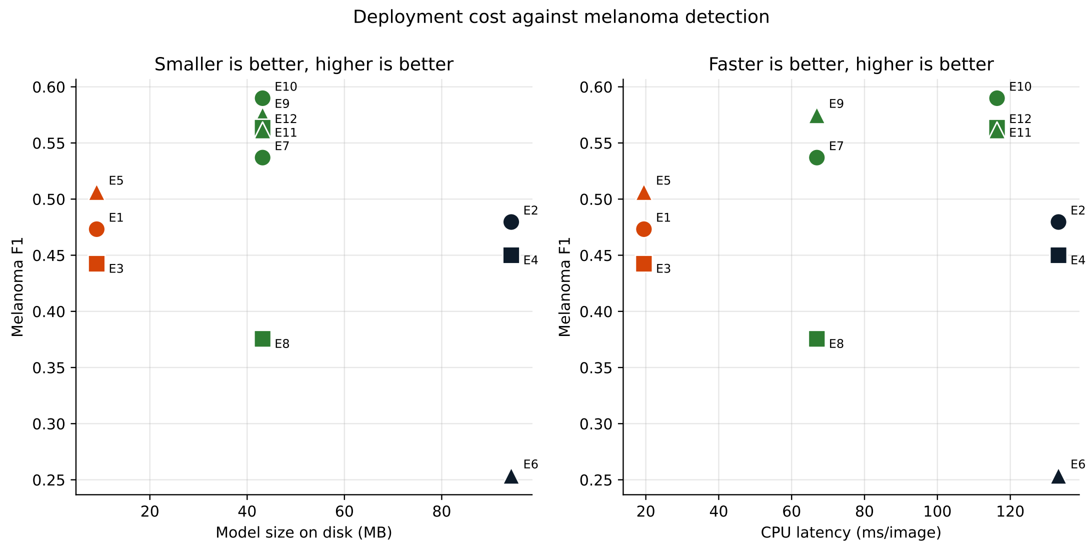

Figure 7.10: Deployment cost against melanoma detection. Left: model size on disk. Right: CPU inference latency. Points toward the upper left are preferable on both panels.

ResNet-50 and EfficientNet-B3 cannot be ranked against each other without naming a CPU. On the ARM64 host, which is the architecture mobile devices actually use, ResNet-50 is 2.9 times faster than EfficientNet-B3; on x86-64 it is 2.0 times slower.

Combined with RQ1, where the melanoma F1 gap between the best MobileNetV2 and best ResNet-50 configurations was 0.037, the trade-off is asymmetric in the lightweight model's favour: roughly a tenth of the size and half the latency, for a difference in melanoma F1 inside the threshold set in advance.

Chapter 8

Discussion

8.1 The Headline Result

MobileNetV2 achieved higher macro F1 than ResNet-50 under all three imbalance strategies, and under two of the three the two architectures were statistically indistinguishable on per-image correctness. A network holding 2.23 million trainable parameters was never beaten by one holding 23.52 million, on the same data, through the same pipeline, with the same optimiser and schedule.

The temptation is to read this as MobileNetV2 being the better architecture. That reading is too strong, and the more defensible one is more interesting: at this dataset size, capacity beyond MobileNetV2's was not the binding constraint. HAM10000 provides 8,012 training images across seven classes, of which the three rarest contribute 467 between them. ResNet-50 has roughly ten times the parameters to fit from the same evidence, and the extra capacity had nothing useful to consume. What it did instead is the subject of Section 8.2.

That distinction matters for how far the result generalises. It supports the claim that a lightweight backbone is sufficient for HAM10000-scale dermatoscopic classification. It does not support a claim that MobileNetV2 would remain competitive given an order of magnitude more data, and nothing here tests that.

8.2 ResNet-50's Optimisation Behaviour

The ResNet-50 runs did not descend smoothly. Measuring the standard deviation of the epoch-to-epoch change in validation loss, averaged within each architecture, gives 0.130 for MobileNetV2 and 0.221 for ResNet-50, a factor of 1.70. E4 produced a single-epoch increase in validation loss of 1.207 and moved in the wrong direction on 23 of its 49 epoch transitions, close to half.

This is not a defect in the implementation. Every relevant source file was reviewed and the behaviour is consistent with a known sensitivity: a deep, BatchNorm-heavy network fine-tuned with Adam at 10^{-3}, with no warmup, no learning-rate schedule and no weight decay. Section 5.5.1 records those omissions, and they are the most plausible cause. MobileNetV2 is shallower and its inverted-residual blocks appear more forgiving of the same configuration.

The learning rate was deliberately not lowered for ResNet-50 alone. Doing so would have tuned one arm of a comparison whose entire premise is that only the architecture varies, and the resulting numbers would have answered a different question than the one asked. The instability is therefore reported as a finding.

It is a finding with practical weight. A network that trains reliably at a shared learning rate is cheaper to work with than one requiring its own schedule, and that cost is real even though it appears in no accuracy table. Combined with the efficiency results, it strengthens rather than complicates the case for the lightweight model.

8.3 Early Stopping Chose the Wrong Checkpoints

The protocol stops on validation loss and evaluates the checkpoint with the lowest validation loss. Validation loss is not the reported metric. Macro F1 is, and the two do not peak at the same epoch.

Table 8.1: Macro F1 at the selected checkpoint against the best macro F1 reached during training. The final column is what checkpoint selection on validation loss cost.

Exp	Best val-loss epoch	Macro F1 there	Best macro F1 epoch	Best macro F1	Cost
E1	28	0.744	28	0.744	0.000
E2	36	0.640	29	0.672	0.032
E3	24	0.613	30	0.650	0.038
E4	46	0.511	49	0.571	**0.059**
E5	20	0.721	30	0.724	0.003
E6	6	0.534	11	0.601	**0.067**

E6 is the clearest case. Its validation loss reached its minimum at epoch 6 and never improved again, so patience expired at epoch 16 and the epoch-6 weights were the ones evaluated on test. Validation macro F1 at epoch 6 was 0.534. By epoch 11 it had reached 0.601 and it was still above 0.59 at the point training stopped. The reported test results for E6, including its melanoma recall of 0.157, come from a model taken ten epochs before its best measured configuration.

This is a limitation of the protocol rather than of ResNet-50, but it does not fall evenly. The cost is 0.000 and 0.003 for two of the three MobileNetV2 runs and 0.059 and 0.067 for two of the three ResNet-50 runs. The reason is the instability of Section 8.2: a noisy validation loss curve has an argmin that lands somewhere arbitrary, whereas a smooth one has an argmin near the genuine optimum.

Part of the gap this thesis reports between the two architectures is therefore attributable to the checkpoint selection rule interacting with optimisation noise, not to the architectures themselves. The direction of the effect favours MobileNetV2, which is the model the thesis concludes for, so this is a caveat that cuts against its own conclusion and is stated for that reason.

The protocol was not changed after this was discovered. Changing the selection rule mid-study, after seeing which runs it disadvantaged, would be a considerably worse methodological error than the one it corrects. Section 9.4 records selection on validation macro F1 as the first item of future work.

8.4 Why the Imbalance Remedies Hurt ResNet-50

RQ2 produced an asymmetric answer. Weighted cross-entropy improved MobileNetV2's clinical recall criterion from 0.589 to 0.648, while both remedies reduced ResNet-50's: from 0.516 to 0.378 under weighted cross-entropy and to 0.352 under oversampling. Interventions designed to help rare classes made one architecture measurably worse at those classes.

The explanation follows from the two preceding sections. Both remedies increase gradient variance. Weighted cross-entropy multiplies the loss on a dermatofibroma example by 12.578 against 0.214 for a nevus, a ratio of 58.8 (Table 4.4), so a single rare example can dominate a batch. Oversampling achieves a similar effect by repetition. On a network already optimising unstably at the shared learning rate, added gradient variance compounds the problem rather than being absorbed by it.

E4 is the endpoint of that interaction: ResNet-50's sensitivity meets the most aggressive reweighting, it never converges within 50 epochs, and it produces the weakest result in the study. Its basal cell carcinoma recall of 0.208 is the lowest figure in the entire factorial, on a class it was explicitly reweighted to favour.

The general claim this supports is narrow and worth stating precisely: imbalance remediation is not architecture-neutral. Its effect depends on how the underlying optimisation behaves, and a remedy that helps one backbone can harm another under otherwise identical conditions. Gap 2 of Section 3.1 asked whether the best strategy depends on the architecture. It does, and the mechanism is optimisation stability rather than anything about the classes themselves.

8.5 Precision, Recall, and What Should Be Deployed

Table 7.3 shows configurations distributed along a trade-off curve rather than ordered from worse to better. E3 finds 63.5% of melanomas and is correct on 34.0% of its melanoma calls. E6 is correct on 66.7% of its melanoma calls and finds 15.7% of melanomas. On 115 test melanomas that is 73 found against 18.

Neither is unambiguously better in the abstract, which is exactly why the criterion was fixed in advance rather than chosen once the numbers existed. Under a screening workflow where flagged lesions receive specialist review, a false negative is an untreated cancer and a false positive is a consultation. E3's 42 missed melanomas against E6's 97 is the comparison that matters, and it is not close.

The honest qualification is that this reasoning depends on the downstream workflow. A tool whose outputs are acted on without review, or one deployed where referral capacity is the binding constraint, would weight precision differently. The criterion encodes an assumption about deployment context, and it should be re-derived rather than inherited if that context changes.

8.6 Comparison with Published Work

The accuracy figures here sit below much of the HAM10000 literature. Gao et al. [9] report 98.97%; Le et al. [12] report 93% averaged across classes. The best accuracy in this study is 0.801.

Direct comparison is not meaningful, for reasons established in Section 2.2. Reported HAM10000 accuracy depends heavily on whether the split is at lesion or image level, and studies frequently do not state which. An image-level split places photographs of the same lesion on both sides of the boundary, and on a dataset where 1,956 of 7,470 lesions are photographed more than once, that inflates the result by an amount no reader can estimate from the paper. The lesion-level protocol used here is the stricter choice and produces lower, better-founded numbers.

The comparison this thesis can make is internal, which is the point of running everything through one pipeline. All twelve configurations share a split, an augmentation pipeline, an optimiser, a schedule and a GPU. Differences between them are attributable to the factors that varied, and that is a claim none of the cross-paper comparisons in the surveyed corpus can make.

8.7 What the Ablation Adds, and What It Does Not

EfficientNet-B3 produced the highest-scoring run in the study. E11 reaches macro F1 0.738 against the best confirmatory run's 0.681, and it does so while holding 4.80 times MobileNetV2's parameters. A thesis arguing for the lightweight model has to deal with that honestly rather than around it.

Three things make it less awkward than it first appears.

The first is that at matched input resolution, B3 and MobileNetV2 are not statistically distinguishable. All three of those comparisons fail to reject after Holm correction, two of them at $p = 1.000$. B3's headline advantage comes from E11, which is a 300-pixel run, and the resolution pairs show that the 300-pixel input is what carries it: E8 to E11 is the single largest and only significant resolution effect in the ablation. The architecture is not doing the work; the extra pixels are.

The second is that E11's advantage is bought with exactly the resource the thesis is about. At 300 pixels B3 costs 8.9 times MobileNetV2's inference time on ARM64 and 4.8 times its disk footprint. For a screening tool that runs on a phone, that is not a favourable trade at any accuracy. The question this thesis asks is what a deployable architecture gives up, and B3 answers a different question well.

The third is a caution about the ablation's own status. E7 to E12 were added after the confirmatory design was fixed and run. They are analysed in a separate family for that reason (Section 6.6.2), and no amount of good performance changes the fact that they were not pre-registered. Treating E11 as the thesis's headline result would mean promoting a post-hoc addition over the design that was specified in advance, which is the precise failure mode the two-family structure exists to prevent.

What the ablation does establish is worth stating plainly. A third architecture, of intermediate size and from a different design family, lands in the same place as the other two: indistinguishable from a much smaller network at matched input size. Three architectures spanning 2.2 to 23.5 million parameters perform comparably on this dataset, which is stronger evidence for the capacity argument of Section 8.1 than the two-architecture comparison could provide alone.

It also produces the one clean methodological result in the ablation: input resolution matters for B3 only under weighted cross-entropy, the highest-variance training condition in the study. That is a finding the published literature cannot report, because it varies architecture and resolution together.

8.8 Limitations

Checkpoint selection. Documented in Section 8.3. It disadvantages the noisier architecture and contributes an unknown fraction of the reported architecture gap. This is the most consequential limitation in the thesis.

Single seed. Every configuration was trained once. Run-to-run variance under a different seed is therefore unmeasured, and small differences between configurations cannot be distinguished from seed noise. The McNemar tests address whether two *specific trained models* differ, which is a narrower claim than whether two configurations differ in expectation. Multiple seeds per cell would cost twelve times the compute and would be the single most valuable addition to this design.

E4 did not converge. It exhausted the 50-epoch budget with validation loss still improving. Its numbers describe an unconverged model and are reported as such.

Unmatched dropout. MobileNetV2 carries $p = 0.2$, EfficientNet-B3 $p = 0.3$ and ResNet-50 none (Section 5.3.1). The asymmetry favours the models with dropout, one of which the thesis concludes for.

No weight decay or schedule. Uniform across runs and therefore not a confound between conditions, but the probable cause of the instability that shaped several results.

Rare-class sample sizes. Dermatofibroma has 13 test images and vascular lesions 15. A single prediction moves dermatofibroma recall by 7.7 percentage points. Per-class figures for these classes are noisy and should not carry argumentative weight.

Single dataset. HAM10000 only, by design (Gap 1). Nothing here establishes that the ranking transfers to another dataset or another acquisition protocol.

Demographic coverage. HAM10000 carries no Fitzpatrick skin-type annotation and is drawn from two predominantly light-skinned populations. Whether these results hold across skin tones is untested and untestable on this data. For a tool intended to widen access to screening, this is a serious gap rather than a technical footnote.

No qualitative analysis. No Grad-CAM or misclassification review was performed, so there is no evidence about whether the models attend to lesions rather than to acquisition artefacts such as hair, rulers or ink markings.

Chapter 9

Conclusion and Future Work

9.1 Answers

RQ1 passes. The best MobileNetV2 configuration reaches a melanoma F1 of 0.442 against 0.480 for the best ResNet-50 configuration, a gap of 0.037 against a threshold of 0.05 fixed before any run executed. The result holds under the alternative definition of "best" as highest macro F1, where the gap narrows to 0.027 and reverses direction. A network roughly a tenth the size gives up nothing measurable on the class that matters most.

RQ2 is architecture-dependent. Weighted cross-entropy is best for MobileNetV2 (E3), augmentation-only for ResNet-50 (E2), where both remedies made matters worse rather than better. There is no single best imbalance strategy for this dataset; there is a best strategy per backbone, and Chapter 8 traces the difference to optimisation stability rather than to anything about the classes.

RQ3 favours MobileNetV2 on every measure. It is the smallest at 9.07 MB against ResNet-50's 94.30 MB, holds 10.5 times fewer parameters, and is the fastest of the three on both CPU architectures tested.

The unanticipated part is that parameter count did not predict latency, and that the ranking of the two heavier models was not even stable across hosts. EfficientNet-B3 holds 2.2 times fewer parameters than ResNet-50 yet runs 2.9 times slower on ARM64 and 2.0 times faster on x86-64. Depthwise separable convolutions trade arithmetic volume for memory traffic, and whether that trade pays depends on where the host binds.

Two things follow. Efficiency reported as parameter count, which is what the literature usually reports, does not predict what a model costs to run. And an efficiency claim that does not name the CPU it was measured on is not a claim at all. MobileNetV2's advantage is the only part of the picture robust to that choice, which is what makes it the defensible deployment recommendation.

The ablation does not change the recommendation. EfficientNet-B3 produced the highest-scoring run in the study (E11, macro F1 0.738), but at matched input resolution it is statistically indistinguishable from MobileNetV2 in all three comparisons, and its advantage traces to the 300-pixel input rather than to the architecture. At that resolution it costs 8.9 times MobileNetV2's inference time on ARM64. It is the better choice for a server-side deployment and the worse one for the offline mobile setting this thesis is about.

9.2 Contributions

The first is the controlled factorial itself. Every configuration shares a split verified by hash, an augmentation pipeline, an optimiser, a schedule and a GPU, so differences between cells are attributable to the two factors that varied. No published HAM10000 study identified in Chapter 2 makes that claim, and it is what allows the RQ2 finding to be stated as a property of the architectures rather than an artefact of two papers using different setups.

The second is per-class reporting throughout. Melanoma F1 varies by a factor of two across the six confirmatory runs, from 0.254 to 0.506, while overall accuracy varies by 0.10 and, between E1 and E2, by 0.004. Any study reporting only the aggregate would present those two configurations as near-equivalent.

The third is the finding that imbalance remediation is not architecture-neutral. Weighted cross-entropy helped MobileNetV2 and harmed ResNet-50 under otherwise identical conditions. A recommendation of an imbalance strategy without reference to the backbone is therefore incomplete.

The fourth is diagnostic rather than experimental. Chapter 8 documents that checkpoint selection on validation loss cost up to 0.067 macro F1, concentrated on the architecture with the noisier optimisation. That is a limitation of a protocol this thesis inherited and followed, and identifying it is more useful to anyone repeating this work than the headline numbers are.

The fifth is the resolution ablation. Running EfficientNet-B3 at both 224 and its native 300 pixels, with everything else fixed, separates architectural merit from input size. Published comparisons vary the two together and therefore cannot attribute an EfficientNet advantage to either. The answer here is that resolution contributes materially in one of three training conditions and negligibly in the other two.

The sixth is the implementation. Every table and figure is generated from stored run artefacts by script. A number in this document that disagreed with the experiments would require the generating code to be wrong rather than a typo to have slipped through.

9.3 What This Does Not Establish

Each configuration was trained once. The McNemar tests establish that two specific trained models differ on per-image correctness; they do not establish that two configurations differ in expectation across seeds. Every claim here should be read with that scope.

E4 did not converge within its epoch budget. The findings are HAM10000-specific by design. The dataset carries no skin-tone annotation, so nothing here speaks to performance across Fitzpatrick types, which for a screening tool is a limitation of consequence rather than of completeness.

9.4 Future Work

Select checkpoints on the reported metric. Section 8.3 shows selection on validation loss cost up to 0.067 macro F1 and did so unevenly across architectures. Repeating the factorial with selection on validation macro F1 is a small change that would remove a confound affecting the headline comparison. This is the first thing to do.

Multiple seeds. Three to five seeds per cell would separate genuine differences from run-to-run variance and would let the analysis report confidence intervals instead of point estimates. It costs a linear multiple of the compute and is the largest single improvement available to this design.

A regularised re-run. No weight decay, schedule or gradient clipping was used. Repeating with a non-zero L_2 penalty and a short warmup would test whether ResNet-50's instability is architectural, as Chapter 8 argues, or an artefact of an under-regularised optimiser configuration. The current design cannot separate the two.

Matched dropout. Adding $p = 0.2$ before ResNet-50's classifier would quantify how much of MobileNetV2's margin comes from regularisation rather than architecture. It is a one-line change.

Extend the resolution ablation to the other backbones. The B3 pairs showed that input resolution matters only in combination with weighted cross-entropy (Section 7.10). Whether that interaction is a property of EfficientNet or of the training condition is not answerable from one architecture. Running MobileNetV2 and ResNet-50 at a second resolution would separate the two.

Skin-tone-stratified evaluation. On a dataset carrying Fitzpatrick annotations, or against an inferred estimate. Until this is done, no deployment recommendation from this work should be read as applying across skin tones.

Qualitative error analysis. Grad-CAM [19] on the recommended configuration, and a misclassified-image review for the malignant classes, to check the models attend to lesions rather than to acquisition artefacts.

On-device measurement. Latency here is measured on a desktop CPU. A figure from an actual mobile SoC would speak directly to the deployment claim, which desktop CPU timing only approximates.

9.5 Closing

The question was what a lightweight architecture costs on the classes a screening tool cannot afford to miss. On HAM10000, under a controlled protocol, the answer is that it costs 0.037 of melanoma F1 against a ten-fold reduction in parameters, and that under two of three imbalance strategies the two architectures cannot be told apart on per-image correctness at all.

The more useful result is the one that was not anticipated. The best imbalance strategy turned out to depend on the architecture it was applied to, and the mechanism was optimisation stability rather than class frequency. That is not visible in any study that varies one factor at a time, and it is the kind of thing a controlled factorial exists to find.

Bibliography

[1] R. S. Aa, V. Chamola, Z. Hussain, F. Albalwy, and A. Hussain, "A novel end-to-end deep convolutional neural network based skin lesion classification framework," *Expert Systems with Applications*, vol. 246, p. 123 056, 2024, Source: Reference Papers/1-s2.0-S0957417423035583-main.pdf. DOI: `10.1016/j.eswa.2023.123056` (cit. on p. ix).

[2] M. Alruwaili and M. Mohamed, "An integrated deep learning model with EfficientNet and ResNet for accurate multi-class skin disease classification," *Diagnostics*, vol. 15, no. 5, p. 551, 2025, EfficientNet+ResNet fusion branches. Source: diagnostics-15-00551.pdf. DOI: `10.3390/diagnostics15050551` (cit. on p. xvi).

[3] M. Buda, A. Maki, and M. A. Mazurowski, "A systematic study of the class imbalance problem in convolutional neural networks," *Neural Networks*, vol. 106, pp. 249–259, 2018, arXiv:1710.05381. The standard reference for oversampling vs. re-weighting on imbalanced CNN training. Verified via arXiv 2026-04-28. [Online]. Available: `https://arxiv.org/abs/1710.05381` (cit. on pp. x, xxviii).

[4] T.-M. Chiu, I.-C. Chi, Y.-C. Li, and M.-H. Tseng, "Deep ensemble learning for multiclass skin lesion classification," *Bioengineering*, vol. 12, no. 9, p. 934, 2025, CSMUH dataset; 25 pretrained models. Source: bioengineering-12-00934-v2.pdf. DOI: `10.3390/bioengineering12090934` (cit. on p. xvi).

[5] N. C. F. Codella et al., "Skin lesion analysis toward melanoma detection 2018: A challenge hosted by the international skin imaging collaboration (ISIC)," 2019, arXiv:1902.03368. Documents the ISIC 2018 challenge (which used HAM10000 as the training set). Verified via arXiv 2026-04-28. arXiv: `1902.03368 [cs.CV]`. [Online]. Available: `https://arxiv.org/abs/1902.03368` (cit. on p. viii).

[6] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:1901.05555. Theoretical jus-

tification for inverse-frequency / effective-number weighting in cross-entropy. Verified via arXiv 2026-04-28., 2019. [Online]. Available: https://arxiv.org/abs/1901.05555 (cit. on p. x).

[7] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998, Standard reference recommending McNemar's test for comparing two classifiers on a single test set; this thesis uses it for the 2x3 paired comparisons. Verified via MIT Press 2026-04-28. DOI: 10.1162/089976698300017197 [Online]. Available: https://direct.mit.edu/neco/article-abstract/10/7/1895/6224/ (cit. on pp. xi, xxxvi).

[8] A. Esteva et al., "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, no. 7639, pp. 115–118, 2017, Landmark CNN-vs-dermatologist comparison on 129,450 clinical images. Verified via Nature 2026-04-28. DOI: 10.1038/nature21056 [Online]. Available: https://www.nature.com/articles/nature21056 (cit. on p. iv).

[9] J. Gao, L. Sun, and P. Zhang, *Skin-LiteNet: A lightweight convolutional module for skin disease image classification*, HAM10000: 98.97% acc, 98.89% F1, 284.44K params. Source: Skin-LiteNet PDF, 2024 (cit. on pp. ix, lix).

[10] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:1512.03385. ILSVRC 2015 winning architecture. ResNet-50 used as the heavyweight comparator in this thesis. Verified via arXiv 2026-04-28., 2016, pp. 770–778. [Online]. Available: https://arxiv.org/abs/1512.03385 (cit. on pp. iv, ix).

[11] A. G. Howard et al., *MobileNets: Efficient convolutional neural networks for mobile vision applications*, arXiv:1704.04861. Introduces depthwise-separable convolutions and the MobileNet family. Verified via arXiv 2026-04-28., 2017. arXiv: 1704.04861 [cs.CV]. [Online]. Available: https://arxiv.org/abs/1704.04861 (cit. on p. ix).

[12] D. N. T. Le, H. X. Le, L. T. Ngo, and H. T. Ngo, *Transfer learning with class-weighted and focal loss function for automatic skin cancer classification*, arXiv:2009.05977. Modified ResNet-50 with class-weighted and focal loss on HAM10000; reports 93% top-1 accuracy. Closest published prior to the imbalance-strategy axis of this thesis. Verified via arXiv 2026-04-28., 2020. arXiv: 2009.05977 [cs.CV].

[Online]. Available: https://arxiv.org/abs/2009.05977 (cit. on pp. x, xi, xvi, lix).

[13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, arXiv:1708.02002. Focal loss; cited in this proposal as the imbalance method explicitly out-of-scope. Verified via arXiv 2026-04-28., 2017. [Online]. Available: https://arxiv.org/abs/1708.02002 (cit. on p. x).

[14] I. Manole, A.-I. Butacu, R. Bejan, and G.-S. Tiplica, “Enhancing dermatological diagnostics with EfficientNet: A deep learning approach,” *Bioengineering*, vol. 11, no. 8, p. 810, 2024, 95.4% validation accuracy (4 classes). Source: bioengineering-11-00810.pdf. DOI: 10.3390/bioengineering11080810 (cit. on pp. v, x).

[15] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, arXiv:1912.01703. Implementation framework for the experiments. Verified via arXiv 2026-04-28., 2019. [Online]. Available: https://arxiv.org/abs/1912.01703 (cit. on p. xxx).

[16] A. Rezvantalab, H. Safigholi, and S. Karimijeshni, *Dermatologist level dermoscopy skin cancer classification using different deep learning convolutional neural networks algorithms*, Source: Reference Papers/1810.10348v1.pdf (filename misleading), 2020 (cit. on p. ix).

[17] O. Russakovsky et al., “ImageNet large scale visual recognition challenge,” *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015, arXiv:1409.0575. Source of the ImageNet pretraining used to initialise both backbones. Verified via arXiv 2026-04-28. DOI: 10.1007/s11263-015-0816-y [Online]. Available: https://arxiv.org/abs/1409.0575 (cit. on p. xxii).

[18] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, arXiv:1801.04381. CVPR 2018 pp. 4510–4520. The exact lightweight architecture evaluated in this thesis. Verified via arXiv 2026-04-28., 2018, pp. 4510–4520. [Online]. Available: https://arxiv.org/abs/1801.04381 (cit. on pp. v, ix).

[19] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*,

arXiv:1610.02391. Used in this thesis only as an optional qualitative tool for the error-analysis section. Verified via arXiv 2026-04-28., 2017. [Online]. Available: `https://arxiv.org/abs/1610.02391` (cit. on pp. xvi, lxvi).

[20] Shams and co-authors per JCMR 2025 PDF, "Skin disease classification: A comparison of ResNet, MobileNet, and EfficientNet," *Journal of Current Multidisciplinary Research*, vol. 1, no. 1, pp. 1–7, 2025, DermNet; 93/94/99% for ResNet50/MobileNet/Effi B0. Source: JCMR PDF. DOI: `10.21608/jcmr.2025.327880.1002` (cit. on pp. ix, x).

[21] P. Tschandl, *HAM10000 dataset (harvard dataverse replication data)*, Harvard Dataverse, Canonical download for the HAM10000 images and ground-truth labels. Verified 2026-04-28., 2018. DOI: `10.7910/DVN/DBW86T` [Online]. Available: `https://doi.org/10.7910/DVN/DBW86T` (cit. on p. xviii).

[22] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, p. 180 161, 2018, 10,015 dermatoscopic images, 7 diagnostic classes. Verified via Nature 2026-04-28. DOI: `10.1038/sdata.2018.161` [Online]. Available: `https://www.nature.com/articles/sdata2018161` (cit. on pp. viii, xviii).

[23] M. Wei, Q. Wu, H. Ji, J. Wang, et al., "A skin disease classification model based on DenseNet and ConvNeXt fusion," *Electronics*, vol. 12, no. 2, p. 438, 2023, HAM10000: 95.29% acc, 89.99% F1. Source: electronics-12-00438-v2.pdf. DOI: `10.3390/electronics12020438` (cit. on p. xvi).